

研究計画 / 2026 年 10 月 7 日 / v1.4

固定行列の剰余表現再利用と 収束に応じた適応精度による 局在基底 DFT の GPU 高速化

Convergence-aware adaptive precision and reusable residue
representations for GPU-accelerated localized-basis DFT

対象：OpenMX + GEMMul8 開発：kugui / NVIDIA A100 評価：GeForce RTX 5080

v1.2 の中心仮説 SCF の進行に応じた GEMM 精度の選択と、固定された直交化行列の前処理再利用を組み合わせることで、最終精度を満たす固定設定よりも必要精度への到達時間を短縮できる。

第 1 段階 (A100) と RTX 5080 の第 1 回測定の結論 (v1.3) 前処理の再利用で減らせる時間は二つの GEMM の 2~8%、適応精度は固定設定を上回らず、二つの GEMM が密行列解法に占める割合は全量確保後で 3~10% にとどまった。両 GPU とも中心仮説を支持する結果は得られず、「全量確保 + fast mode の固定設定」を実用上の結論として、対象を密行列解法の 9 割を占める固有値解法へ移した。

第 2 段階 (2026 年 10 月 6~7 日)：固有値解法の FP32 化と補正 (第 4 節)

- すべての固有対を FP32 で解き、必要な MaxN 個を荻田・相島の反復改良 (積は GEMMul8) で 2 回補正し、部分占有の縮退クラスターをレイリー・リッツで回転する。前のステップの固有ベクトルから補正を始めるウォームスタートを加えた。
- A100 の sidia333 (非共線、 $2n = 5616$) で、全エネルギーは FP64 と 10^{-12} Ha の桁で一致し、SCF 反復は 23 回から 22 回になった。固有値解法の段階は 1 ステップ 873 ms から 140 ms になった。
- RTX 5080 (18 ランク) では、全計算が 261.3 秒から 173.9 秒 (中性、反復 23 回→21 回)、電荷 -2 で 236.1 秒から 173.7 秒になり、固有値解法の段階は 1 ステップ 3,189 ms から 163 ms になった。残る時間の大半は Set_Hamiltonian (79 秒、1 ステップ 3.8 秒) と Set_Density_Grid (33 秒) である。
- 同じ補正を共線クラスター (スピント)、共線バンド、非共線バンド (1 ランク 1k 点の経路) に広げ、A100 で検証した。GeForce など FP32:FP64 比が 8 以上の GPU では既定で有効にした。

v1.4 は、v1.3 の方針転換 (固有値解法への移行) を受けて 2026 年 10 月 6~7 日に実装・検証した補正つき FP32 固有値解法と、その結果を反映した改訂である。v1.3 までの内容 (GEMM の再利用と適応精度) は第 2~8 節にそのまま残し、固有値解法は第 4 節に新設した。変更点は第 13 節にまとめる。

1 目的、位置づけ、研究質問

1.1 研究対象と精度の意味

本研究は、OpenMX 4.0 GPU 版 [7] の直交化変換に現れる二つの GEMM を対象とする。対象 commit を監査した結果、現行実装の設定は次のとおりであった。v1.2 が開始設定とした $L = 20$ は、このリポジトリの履歴には存在しない。

- moduli 数の既定は $L = 15$ 、fast mode off (accurate)。指定範囲は 2~20 (OPENMX_GEMM8_NUM_MOD_D/Z、OPENMX_GEMM8_FASTMODE_D/Z)。
- 省メモリモードが既定で有効（作業領域の上限 256 MiB、OPENMX_GEMM8_MAX_WORKSPACE_MB）。上限を超える積は分割して実行される。
- 組み込まれている GEMM8 はタグ v3.5.2 (commit 603b523) で、2026 年 10 月 5 日時点の最新タグである。upstream の main は README だけを変更した commit が一つ先行している。

GEMM8 の公開 README が示す有効ビット数は固定小数点エミュレーションの目安であり、出力の精度を保証するものではない [1, 2]。v1.2 はこの点を研究課題とした。第 1 段階では、SCF から採取した実行列について倍々精度の参照値との差を測り、moduli 数と達成精度の関係を得た（第 2 節）。以後の精度の議論はこの実測値に基づく。

主対象は INT8 バックエンド、単一 GPU 上の既存の密行列経路とする。実装と測定は、実行列を扱う共線スピンのクラスター解法から始めた。FP8、逆変換の適応精度化、部分キャッシュ、多 GPU 分散、Kokkos 化は初期成果の必須条件に含めない。

1.2 既存機能・先行研究との差分

公開版 GEMM8 には、前処理済みデータを保持する skip_scalA/B が既にある。再利用には入力内容と演算条件の不変性が必要で、内容の一致は呼出し側が保証する。省メモリモードでは skip による再利用が無効になる [1]。従って、量子化結果を再利用する機能そのものの初出は主張しない。

Huang らは PySCF の密度フィッティングによる交換行列構築で、SCF 中のエネルギー変化に応じて精度を単調に引き上げ、最後に通常の FP64 へ戻す方式を報告している [3]。適応精度・単調な昇格・最終 FP64 への移行は、いずれも本研究単独の新規性とはしない。

本研究の差分は、局在基底 DFT の一般化固有値問題における固定直交化行列に着目し、moduli 制御と準備済み表現の寿命管理を結び付ける点にある。第 1 段階の結果から、貢献として確実に示せるのは次の四つである。(1) 実行列での精度・時間・容量の測定と、精度を下げられる範囲。(2) 再利用が厳密に成立する条件（スケーリング方式への依存）と、その削減上限。(3) 作業領域の確保方針が再利用や適応化より大きく効くという事実。(4) 適応精度が固定設定を上回らなかったという結果と、その理由（順変換の比重の小ささと、反復の増加）。RTX 5080 の測定（第 3 節）を受けて、準備済み行列 API の提案と適応精度の研究は打ち切ることを提案する。

	研究質問	答え (A100 と RTX 5080)
RQ1	各 SCF 段階でどこまで精度を下げられるか	全反復を $L = 12$ fast に固定しても反復数は増えず、エネルギー差は FP64 自身の揺らぎ (5×10^{-11} Ha 以下) の範囲。 $L = 10$ は 0~4 回増、 $L = 8$ は最大 48 回増
RQ2	固定行列の前処理再利用はいつ有効か	fast mode でのみ厳密 (両 GPU)。削減は二つの GEMM の A100 で 2~5 %、RTX 5080 で 4~8 %。保持に 4.9 GiB ($n = 13,000$ 、 $L = 15$)
RQ3	適応化は最良の固定設定より有利か	A100 では有利でなかった (未調整入力で反復が 0~3 回増え、順変換の差は計算全体の 0.1 % 程度)。RTX 5080 では順変換が密行列解法の 3 % 前後しかなく、有利になる余地がない
RQ4 (v1.4 追加)	固有値解法の精度を下げて SCF の精度を保てるか	FP32 の固有対を 2 回の反復改良 (積は GEMMu8) で補正すると、全エネルギーは FP64 と 10^{-12} Ha の桁で一致し、反復数は増えない (A100・RTX 5080、第 4 節)。固有値解法の段階は A100 で 6 分の 1、RTX 5080 で 4 分の 1 (ウォームスタート前) になった

以下では入力のスケーリングと低精度表現への変換を quant と呼ぶ。

1.3 2026 年 10 月 5 日の決定事項

第 1 段階の報告に対して、次の五点を決めた。本版はこれを前提とする。

1. 再利用 API を続けるかどうかは、RTX 5080 での測定を見て判断する。
2. 実装はブランチ adaptive-precision に置く。
3. fast mode を主設定とする。
4. 最終段階の規則 (混合履歴を再開しない、早めに FP64 へ移る、ネイティブ FP64 を一切使わない) は較正で比べて決める。
5. メモリが足りるときは作業領域を全量確保する動作に切り替える。

2 第 1 段階で確定した事実 (kugui / A100)

2.1 環境と入力

ISSP kugui の GPU ノード (NVIDIA A100-SXM4-40GB、NVHPC 24.7、CUDA 12.5、cuBLAS 12.5.3、cuSOLVER 11.6.3) を使った。この cuSOLVER には FP64 のエミュレーションがなく、固有値解法はネイティブ FP64 で動く。MPI は 16 ランク (1000 原子の入力は 24 ランク) で、GPU は MPS を介して共有する。密行列解法は一つのランクが担当する。

入力	基底数 n	用途	性質
GFRAG	702	較正	非磁性、グラフェン片
Pt63	819	較正	金属クラスター
Mn12_148_F	1,088	較正	磁性（スピン分極）
nsV4Bz5	730	評価	$V_4(C_6H_6)_5$ 分子（遷移金属を含む）
DIA512-1	2,048	評価	ダイヤモンド 512 原子。Krylov 法用の入力をクラスター解法で
MCCN	2,820	評価	多重連結カーボンナノチューブ。同上
CG15c、MCCN'	6,530、7,332	時間のみ	SCF を 3 回で打ち切る入力（MCCN' は基底の大きい版）
C1000	13,000	時間・行列	3 回で打ち切る入力と行列採取
Si1000	13,000	評価・行列	収束まで 41 回

現行のクラスター経路では、 S の固有値を 10^{-10} で下から抑えて $X = U_s^{-1/2}$ を作るの、常に $r = n$ の実正方行列である。 X は各構造の最初の SCF で作られ、以後の SCF では同じデバイス上の領域に保持される。

2.2 行列単体：精度

SCF の初期・中期・終盤で H_t と X を採取し、 $C = X^T H_t X$ を倍々精度の参照値（標準列）と比べた。表は C の相対誤差の最大値である（fast mode、再利用なし）。

入力	n	FP64	$L = 8$	$L = 10$	$L = 12$	$L = 14$	$L = 15$	$L = 16$
Pt63	819	9.4e-15	8.4e-8	4.0e-10	1.7e-12	7.0e-15	4.9e-16	1.2e-16
Mn12_148_F	1,088	1.8e-14	6.1e-8	2.7e-10	1.5e-12	6.3e-15	4.2e-16	1.2e-16
GFRAG	702	1.8e-14	3.9e-7	1.4e-9	7.8e-12	3.9e-14	1.9e-15	1.9e-16
C1000	13,000	6.6e-14	6.2e-7	2.3e-9	2.0e-11	7.1e-14	4.3e-15	3.5e-16
Si1000	13,000	6.6e-14	7.3e-7	3.7e-9	2.1e-11	8.0e-14	4.3e-15	4.1e-16

- moduli を 1 個増やすごとに誤差は 1 桁強下がり、 $L = 16$ で参照値との比較の限界に達する。
- $L = 14$ で cuBLAS FP64 と同程度、 $L = 15$ では FP64 より 9~44 倍正確である。「最終段階は FP64 でなければならない」という数値的な理由は、この五つの系には見当たらない。
- 誤差は SCF の段階（初期・中期・終盤）に依存しなかった。
- 採取した H_t 自体が、相対 4×10^{-10} 程度の非対称性を持つ（要素あたり最大 10^{-9} Ha 程度）。 $L \geq 12$ の積の誤差よりはるかに大きい。

2.3 行列単体：再利用の成立条件

同じ X を前処理して保持し、 H_t だけを変えて通常経路と比べた (moduli 数 8~20、他の SCF 反復・他方のスピンの H を含む)。

- **fast mode**：再利用した積は通常経路と全要素がビット単位で一致した。
- **accurate mode**：前処理を行う呼出しは通常経路と一致するが、再利用する呼出しはほぼ全要素が変わり、誤差は 8~33 倍に増えた。moduli を 1 個減らした場合に相当する。

原因は GEMMul8 v3.5.2 のスケーリング規則にある。fast mode は A と B を独立にスケールするので、準備済みの X は X だけで決まる。accurate mode は、再利用する側のシフトを準備時の相手行列と合わせて決めた値に固定し、相手側を呼出しごとに補正する。相手に依存する情報が残るので、v1.2 の第 2.2 節が警告した場合に当たる。

2.4 行列単体：時間と容量

$n = 13,000$ (c1000) での二つの GEMM の合計時間を示す。作業領域を全量確保した「通常」、 X を再利用した場合、256 MiB 上限で分割した場合である。cuBLAS FP64 は 499 ms であった。Si1000 でも同じ値 (差は 1% 以内) が得られた。

L	fast mode (ms)			accurate mode (ms)		
	通常	再利用	分割	通常	再利用	分割
8	190	188	315	217	209	380
10	240	232	405	264	256	474
12	284	276	509	311	301	595
14	330	323	625	357	347	714
15	354	346	698	379	371	784
20	470	461	937	497	484	1,032

- 時間は moduli 数にほぼ比例する (1 個あたり約 23 ms)。 $L = 15$ fast は FP64 の 0.71 倍、 $L = 12$ fast は 0.57 倍である。
- 再利用による短縮は 1.4~3.9% で、表の主要な設定では 2~3% である。スケーリングの時間は全体の 6% (fast)、13% (accurate) しかなく、そのうち X の分だけが再利用で消える。
- 分割実行は全量確保の 1.7~2.1 倍かかる。分割した各ブロックが被演算子をスケールし直すため、スケーリングの時間は 49 ms から 276 ms に増える。分割実行の $L = 15$ は FP64 より遅い。
- 合成行列でも同じ傾向である。再利用の短縮は $n = 4000$ で 5~7%、 $n = 8000$ で 3~4%。スケーリングは $O(n^2)$ 、積は $O(n^3)$ なので、行列が大きいくほど再利用の取り分は減る。

2.5 SCF 内部：密行列解法の内訳

OpenMX の中で、密行列解法 1 回あたりの時間を段階ごとに測った（各段階の前後でストリームを同期、単位は ms）。順変換が対象の二つの GEMM である。「旧既定」は 256 MiB 上限での分割、「新既定」は第 7.2 節の全量確保、「 $L = 15$ fast」は再利用ありである。

入力	n	順変換				固有値	順変換の割合
		FP64	旧既定	新既定	$L = 15$ fast	解法	旧既定 $\rightarrow$ $L = 15$ fast
GFRAG	702	0.24	0.72	0.74	0.6	12	5 % $\rightarrow$ 4 %
DIA512-1	2,048	2.0	3.1	3.0	2.5	42	6 % $\rightarrow$ 5 %
CG15c	6,530	62	106	56	49	628	14 % $\rightarrow$ 7 %
MCCN'	7,332	84	145	76	67	823	14 % $\rightarrow$ 7 %
C1000	13,000	517	810	421	370	3,620	17 % $\rightarrow$ 9 %

- 密行列解法の時間の大半は固有値解法である。順変換の割合は旧既定で最大 17 %、 $L = 15$ fast で 9 % ($n = 13,000$)。
- $n \lesssim 2000$ では、A100 の FP64 のほうが GEMM_{ul}8 より速い。GEMM_{ul}8 が FP64 を上回るのは n が数千を超えてからである。
- 各実行の最初の 1 回には、一度だけの初期化（GEMM_{ul}8 で約 40 ms）と X の前処理が加わる。SCF を 3 回で打ち切る入力では、これを分けないと比較を誤る。集計は 2 回目以降を使う。
- GEMM_{ul}8 の呼出しは、ときどき 30~300 ms 遅れる回がある（MPS 下の A100 で観測。原因は未確認）。FP64 の積では見ていない。平均が動くので、時間の比較には 2 回目以降の中央値も使う。
- OpenMX の中での値は、行列単体の測定より 7~11 % 大きい（FP64 も GEMM_{ul}8 も増える）。GPU が数秒休んだ直後の演算であることが原因と考えているが、確認していない。

2.6 計算全体

- FP64 基準の変動**：同じ入力の FP64 計算を 3 回繰り返すと、全エネルギーは最後の桁まで一致した。`scf.criterion` を 10^{-10} から 10^{-12} に厳しくしたときの変化は、Pt63 と Mn12_148_F で 2×10^{-12} Ha 以下、GFRAG で 3×10^{-11} Ha（力は 1×10^{-10} Ha/Bohr）である。目標の 10^{-10} Ha に対する余裕は GFRAG で 3 倍しかないので、基準は 10^{-12} で作る。
- 固定設定**：順変換だけを固定設定に替えた結果を下の表に示す。 $L = 12$ 以上では反復数は FP64 より増えず、差は基準の変動以下である。
- 全体に占める割合**：Si1000 の全計算（41 反復、24 ランク）は約 1,260 秒で、このうち対角化が 383 秒（OpenMX の計時）である。順変換は、FP64 で 22 秒（1.8 %）、固定 $L = 12$ fast で 13 秒（1.0 %）だった。旧既定では約 33 秒（2.6 %）、全量確保の新既定では約 17 秒（1.4 %）と見積もられる。

入力	SCF 反復数 (括弧内は $ \Delta E $ 、Ha)				
	FP64	$L = 15$ fast	$L = 12$ fast	$L = 10$ fast	$L = 8$ fast
GFRAG	60	60 (0)	60 (0)	60 (2×10^{-11})	73 (7×10^{-10})
Mn12_148_F	61	61 (0)	61 (9×10^{-13})	65 (9×10^{-13})	109 (4×10^{-10})
Pt63 (10^{-9})	70	70 (8×10^{-12})	66 (2×10^{-9})	67 (4×10^{-10})	71 (1×10^{-8})

Pt63 は入力の収束条件 10^{-9} のままの値で、 10^{-9} Ha 程度の差は収束の揺らぎである。較正 (第 8.3 節) では全入力を 10^{-10} にそろえた。

3 RTX 5080 での第 1 回測定

3.1 条件

作業手順書 v1.1 の三つの測定を、README の「PC」で行った (2026 年 10 月 5 日、コミット a350140)。

- GeForce RTX 5080 16 GB (ドライバ 595.91.07)、Core i9-10980XE、CUDA 13.4 (cuBLAS 13.8)、HPC SDK 26.9、GCC 14.2。
- OpenMX は 18 ランクで、MPS を介して GPU を共有した。cuBLAS のエミュレーションに関する環境変数は設定していない。固有値解法 (cuSOLVER) は既定の設定のままである。
- 比較のため、A100 でも同じコミットのバイナリで測定 3 を繰り返した (16 ランク)。

測定 2 (回帰試験) は、RTX 5080 でも 260 項目すべて合格した。

3.2 行列単体（測定 1）

合成行列での二つの GEMM の合計時間（ms）を、A100 の値と並べて示す。

	$n = 4000$		$n = 8000$	
	A100	RTX 5080	A100	RTX 5080
cuBLAS FP64	17.6	315.7	114.6	2,457
$L = 15$ fast、全量確保	13.3	15.8	90.8	95.8
$L = 15$ fast、再利用	12.5	14.8	88.0	91.4
$L = 15$ fast、分割（256 MiB）	19.1	22.2	153.2	164.9
$L = 12$ fast、全量確保	10.7	12.7	72.4	78.0
再利用による短縮（ $L = 15$ fast）	5.4 %	6.1 %	3.1 %	4.6 %
分割／全量確保	1.44	1.41	1.69	1.72
GEMMul8 / FP64	0.75	0.050	0.79	0.039
スケーリングの割合	12.9 %	13.7 %	8.0 %	9.8 %

- cuBLAS FP64 は、RTX 5080 で A100 の 18～21 倍遅い。GEMMul8 は A100 とほぼ同じ速さ（1.05～1.2 倍）で、RTX 5080 では FP64 の 20～26 倍速い。
- 再利用による短縮は 4.6～6.1 %（A100 は 3.1～5.4 %）、スケーリングの割合は 10～14 % で、どちらも目安（短縮 10 %、スケーリング 20 %）に届かない。
- 分割実行は全量確保の 1.4～1.7 倍で、A100 と同じである。
- fast mode の再利用は、RTX 5080（CUDA 13.4）でもすべての L で通常経路とビット単位で一致した。

3.3 OpenMX の中 (測定 3)

密行列解法 1 回あたりの時間 (ms) を示す。順変換は 2 回目以降の中央値、固有値解法は平均である。旧既定は 256 MiB 上限での分割 (blocked)、新既定は全量確保 (B15a、コミット 2c9cdcd 以降の既定と同じ経路)、C15f・C12f は fast mode・再利用ありの $L = 15 \cdot 12$ である。CG15c と MCCN' は SCF を 3 回で打ち切る入力なので、値は 2 回分の中央値である。

入力	n	順変換					固有値 解法	順変換の割合 旧既定→ C15f
		FP64	旧既定	新既定	C15f	C12f		
<i>A100 (16 ランク)</i>								
GFRAG	702	0.24	0.76	0.73	0.56	0.49	12.5	5.6 % → 4.0 %
Pt63	819	0.29	0.86	0.86	0.64	0.60	14.8	5.2 % → 3.8 %
Mn12_148_F	1,088	0.52	2.14	1.92	1.59	1.26	27.9	7.2 % → 5.1 %
DIA512-1	2,048	1.98	3.07	3.07	2.51	2.06	43.0	6.3 % → 5.1 %
CG15c	6,530	62.1	107	56.2	49.1	39.6	630	13.5 % → 6.7 %
MCCN'	7,332	84.5	145	75.9	66.9	54.7	828	14.1 % → 7.0 %
<i>RTX 5080 (18 ランク)</i>								
GFRAG	702	2.61	0.61	0.55	0.48	0.44	18.5	3.1 % → 2.4 %
Pt63	819	3.55	0.70	0.66	0.55	0.50	23.6	2.8 % → 2.2 %
Mn12_148_F	1,088	12.0	1.69	1.68	1.30	1.11	53.1	3.2 % → 2.3 %
DIA512-1	2,048	44.7	3.41	3.37	2.61	2.15	109	2.9 % → 2.2 %
CG15c	6,530	1,401	127	65.1	55.7	44.8	1,750	6.6 % → 3.0 %
MCCN'	7,332	1,946	159	87.2	75.5	60.4	2,252	6.4 % → 3.1 %

- 全量確保の効果は RTX 5080 でも同じで、 $n \approx 7000$ では順変換が 1.8~1.9 倍速くなった。
- RTX 5080 では、FP64 の順変換は密行列解法の 28~46 % を占める ($n \geq 2048$)。GEMMul8 はこれを、旧既定で 3~7 %、新既定で 3~4 % に抑えている。
- 固有値解法は、RTX 5080 で A100 の約 2.7 倍遅く ($n = 7332$ で 2.25 秒)、密行列解法の 9 割以上を占める。このため、GEMMul8 を使う設定どうしの差 (分割と全量確保、再利用、moduli 数) は、RTX 5080 のほうが A100 より小さく見える。
- 再利用 (C15f と B15f) の差は、RTX 5080 の $n \approx 7000$ で 4~5 %、密行列解法の 0.2 % 以下である。
- 精度：GFRAG、Mn12_148_F、DIA512-1 では、FP64 と $L \geq 12$ のすべての設定が同じ反復数で止まり、エネルギー差は 3×10^{-12} Ha 以下だった。 $L = 10$ は Mn12_148_F で 4 回増えた。Pt63 (収束条件 10^{-9}) は 66~70 回で、差は 10^{-9} Ha 程度の収束の揺らぎである。
- 同じ入力の FP64 どうしでも、A100 と RTX 5080 の全エネルギーは 10^{-11} Ha 程度違う (固有値解法と GEMM の実装が異なるため)。差は各 GPU の FP64 に対して測った。

- A100 で見られた GEMM_{ul}8 の呼出しの突発的な遅れは、RTX 5080 では一度も出なかった。A100 でもノードを専有したジョブでは再現しなかったため、他のジョブとノードを共有する環境の影響とみられる。

3.4 判断

第 7.3 節と第 12.2 節で定めた目安に照らすと、次のようになる。

1. **再利用 API** : RTX 5080 でも短縮は二つの GEMM の 4.6~6.1 % で、目安の 10 % に届かない。upstream への API 提案は行わず、OpenMX 側の実装 (OPENMX_GEMMUL8_FORWARD_REUSE) にとどめる。
2. **適応精度** : RTX 5080 では E と F を直接比べていないが、GEMM_{ul}8 の順変換は密行列解法の 3 % 前後しかなく、E 対 F の差が目安の 5 % に届くことはない。A100 の評価では反復の増加も確認された。適応精度の研究は打ち切り、制御の実装は実験用に残す。
3. **固定設定の既定値** : 順変換の既定を $L = 15$ accurate から fast mode に変えると、RTX 5080 で順変換が約 1 割 ($L = 12$ fast なら約 3 割) 短くなるが、密行列解法では 0.3~1 % である。精度は変わらないので変えてもよいが、急ぐ理由はない。
4. **全量確保の他経路への拡張** : 逆変換は RTX 5080 で順変換の 4 分の 1 程度 ($n \approx 7000$ で 17~20 ms) しかなく、効果は小さい。複数 k 点のバンド計算 (複素数、 k 点ごとの順変換) は未測定である。

4 固有値解法の FP32 化と補正

v1.3 の方針 (第 12.2 節の旧版) に従い、2026 年 10 月 6~7 日に密行列解法の固有値解法を対象にした。本節はその方法、実装、A100 での検証、RTX 5080 での測定をまとめる。実装はブランチ adaptive-precision (2026 年 10 月 7 日時点のコミット f2128528 以降) にある。

4.1 方法の選択

密行列解法の時間の 9 割前後は、変換後のハミルトニアン $\tilde{H} = X^\dagger H_t X$ (非共線では $2n \times 2n$ の複素エルミート行列) の固有値解法 (cuSOLVER の xsyevdx) である。RTX 5080 では FP64 の演算器が少ないため、sidia333 ($2n = 5616$ 、必要な固有対 MaxN= 1987) でこの部分が 1 ステップ 3.2 秒かかる。次の二案を比べた。

案 A SCF の前半を FP32 の固有値解法で進め、残差が小さくなったら FP64 へ切り替える。

案 B すべての固有対を FP32 で解き、必要な MaxN 個を反復改良で FP64 相当の精度まで補正する。

案 A は、FP32 の固有ベクトルの誤差 (10^{-6} 程度) が密度行列に残るため、切替えの時点で反復をやり直す必要があり、scf.criterion= 10^{-10} の精度に対して反復が増えた。案 B は、荻田・相島の対称固有値分解の反復改良 [4, 5] が 1 回の改良で誤差を 2 乗のオーダーに落とすので、 10^{-6} の初期誤差が 2 回

で 10^{-12} 以下になる。改良の本体は $n \times n$ 行列と $n \times \text{MaxN}$ の固有ベクトルの積で、これは GEMMul8 ($L = 15$ 、FP64 相当の精度) で実行できる。案 B を採用した。

4.2 アルゴリズム

$A = \tilde{H}$ (エルミート化済み)、 X を FP32 の固有ベクトル (全 n 個)、 X_1 を補正対象の最初の MaxN 列とする。1 回の改良は次のとおりである。

1. $Y = AX_1$ 、 $S = X^\dagger Y$ 、 $G = X^\dagger X_1$ ($R = I - G$) を GEMMul8 で計算する ($n \times n \times \text{MaxN}$ の積が 3 回、更新で 1 回)。
2. 固有値の推定を $\tilde{\lambda}_j = \text{Re } s_{jj} / \text{Re } g_{jj}$ で更新する。
3. 補正 $E_{ij} = (s_{ij} + \lambda_j r_{ij}) / (\lambda_j - \lambda_i)$ を、 $|\lambda_j - \lambda_i| > \delta$ かつ占有数の差 $|f_i - f_j| \geq 10^{-12}$ のときに使い、それ以外は $E_{ij} = r_{ij} / 2$ (正規直交化だけ) とする。 $\delta = 2(\max_{i \neq j} |s_{ij}| + \|A\| \max |r_{ij}|)$ は最初の改良で決め、以後固定する。
4. $X_1 \leftarrow X_1 + XE$ 。

占有数の差で回転を省くのは、同じ占有数の状態どうしの回転は密度行列を変えないからである。これにより、ほぼ縮退した占有状態の対に大きな補正を入れる必要がなくなる。改良を 2 回行った後、推定固有値が 30δ 以内で連なる「クラスタ」のうち、占有数の広がり 10^{-12} 以上のもの (フェルミ準位近傍の部分占有状態) だけをレイリー・リッツで回転する (クラスタの大きさ $m < 256$ はホストの `zhgegv`、それ以上はデバイスの `zhgevd`、上限 $\min(2048, \sqrt{n \text{MaxN}/2})$)。 δ を 2 回目の改良で取り直すと、縮退クラスタの連なりが 1,436 状態にまで膨らむ例があったため、1 回目の値に固定した。

占有数は、FP32 の固有値推定からソルバーと同じ探索 (電子数、電子温度、 k 点の重み) で化学ポテンシャルを求めて計算する。推定の誤差 10^{-6} Ha は、判定の境界 ($|f_i - f_j| \sim 10^{-12}$) にある対の判定しか変えず、その対の密度行列への寄与は 10^{-18} 以下である。

4.3 実装

- **共有モジュール**: `source/eigen_refine_gpu.c` に、実対称・複素エルミートの両方を扱う形で実装した (準備=エルミート化と FP32 解法またはウォームスタート、仕上げ=改良とレイリー・リッツ)。クラスター法・バンド法の各ソルバーは、行列と作業領域を渡す薄いアダプタで呼ぶ。
- **FP32 の解法**: cuSOLVER 12.3.4 以降では二段階アルゴリズム (`cusolverDnXsyevd`、`CUSOLVER_ALG_2`) を使う。RTX 5080 の $2n = 5616$ で、一段階の 638 ms が 323 ms になった。A100 の cuSOLVER 11.6 では一段階のままである。
- **ウォームスタート**: 2 ステップ目以降は、前のステップの補正済み固有ベクトルを X とし、最初の改良の残差 ($\max |s_{ij}|$ 、 $\max |r_{ij}|$) が 2×10^{-5} 以下 (FP32 解法は 4×10^{-6} を残す) なら FP32 の解法を省く。超えたときは FP32 で解き直す。A100 の `sidia333` では 22 ステップ中 20 回が該当し、固有値解法の段階は 706 ms から 140 ms になった。クラスター法だけの既定とし、バンド法では使わない (後述)。

- **段階制御**：すべての SCF ステップを補正つきで解く（1 ステップ目も）。SCF が補正ステップで収束したときは、同じハミルトニアンを FP64 でもう一度解いて密度行列を作り直す（SCF 後のエネルギー密度行列と力が FP64 の固有ベクトルから出るように）。`scf.maxIter` の最後のステップは FP64 で解く。GPU メモリ不足や FP32 解法の失敗は FP64 に戻る（1 ステップ目は一時的、以後はその SCF サイクルの残り）。GEMMul8 の積が cuBLAS の FP64 に退避した場合は、GeForce では補正の意味がないので以後の実行を FP64 に戻す。
- **既定値**：FP32:FP64 の性能比（`cudaDevAttrSingleToDoublePrecisionPerfRatio`）が 8 以上の GPU（GeForce RTX 5080 は 64、RTX PRO 6000 Blackwell も該当。A100 は 2）で、GEMMul8 の積が有効なら既定で有効にする。`OPENMX_EIGEN_REFINE=0` で無効、`= K` で改良回数の指定。
- **密度行列の積化（非共線クラスター）**：密度行列を状態ごとの和ではなく、占有状態の重み付き固有ベクトル $W = \text{diag}(\sqrt{f}) V$ の積 $W^\dagger W$ （GEMMul8）1 回と疎な成分の取り出しで作り、全ランクの総和の代わりにルートのバッファを配る。A100 で 1 ステップ 385 ms から 123 ms、RTX 5080 で 432 ms から 120 ms 程度になった。
- **他の経路への拡張**：共線クラスター（スピニングごとに実対称、2 つのスピニングのオーナーが固有値推定を交換して化学ポテンシャルを求める）、共線バンドと非共線バンド（1 ランク（ k ワールド）1k 点の経路。全オーナーの FP32 推定を集めて化学ポテンシャルを求めてから仕上げる）。1 ランクに複数 k 点を持つ経路、GPU メモリの都合でターンが直列化される場合、XANES・空軌道指定、1 ランクで両スピンを解く場合は FP64 のままである。

4.4 A100 での検証（2026 年 10 月 6～7 日）

kugui の A100（8 ランク、MPS）で、`sidia333` の各経路を FP64 と比べた。`scf.criterion= 10-10`、A100 は FP64 が速いので GEMMul8 の積を使う補正の効果は小さく、ここでは精度と反復数の確認が目的である。

経路 (sidia333)	設定	SCF 反復	全エネルギー (Ha)	FP64 との差	対角化の時間 (s)
非共線クラスター ($2n = 5616$)	FP64	23	-887.309607369118	—	29.6
	補正	22	-887.309607369117	1×10^{-12}	21.7
同、 <code>scf.system.charge=</code> -2	FP64	21	-887.481166961872	—	—
	補正	21	-887.481166961884	1×10^{-11}	20.0
共線クラスター、ス ピン Off ($n = 2808$)	FP64	20	-887.309607369113	—	15.0
	補正	20	-887.309607369116	3×10^{-12}	13.6
共線クラスター、ス ピン On	FP64	20	-887.309607369092	—	26.2
	補正	20	-887.309607369097	5×10^{-12}	25.3
共線バンド、スピン Off ($2 \times 2 \times 2$ 、8k 点)	FP64	19	-887.422830856531	—	40.1
	補正	22	-887.422830856483	5×10^{-11}	41.1
共線バンド、スピン On ($2 \times 2 \times 1$ 、4k 点)	FP64	22	-887.415346166372	—	48.8
	補正	24	-887.415346166357	2×10^{-11}	45.0
非共線バンド ($2 \times 1 \times 1$ 、2k 点)	FP64	23	-887.377746939165	—	62.0
	補正	24	-887.377746939172	7×10^{-12}	61.4

- 非共線クラスターの固有値解法の段階（密行列解法のうち固有値解法だけ）は、1 ステップ 873 ms (FP64) が、補正のみで 706 ms、ウォームスタートを加えて 140 ms になった（最初のステップは全 $2n$ 状態を補正するので 2.9 秒）。
- 非共線バンドの $2 \times 2 \times 2$ (8k 点、8 オーナー) と、スピン On の共線バンドを 16 ランクで走らせる試みは、ジョブのホストメモリ上限 (60 GB) を超えて止まったため、非共線は $2 \times 1 \times 1$ (2k 点)、スピン On は $2 \times 2 \times 1$ (4k 点 $\times 2$ スピン = 8 オーナー) で検証した。バンドの補正つき実行は SCF 反復が FP64 より 1~3 回多い（共線 Off 19 $\rightarrow$ 22、On 22 $\rightarrow$ 24、非共線 23 $\rightarrow$ 24）。軌道の違いは収束末期の混合 (RMM-DIISK) が 10^{-10} 程度のステップごとの差を増幅したもので、全エネルギーの差は 6×10^{-11} Ha 以下である。改良回数を 3・4 回にしても反復数 (22 回) も軌道も変わらないので、

この差は補正の精度ではなく、混合の感度によるものである（クラスターの補正つき実行でも反復は $-2 \sim +0$ 回ずれる）。

- バンドで補正版の SCF 反復が FP64 より 2~5 回増え（A100 の共線バンド、`scf.criterion` = 10^{-12} で FP64 の 23 回に対し 28 回。RTX 5080 でも +2~+5 回）、ステップをまたぐウォームスタートが 2.5×10^{-10} Ha の偏りを持つ原因を突き止めた。GEMM8 の法の数 (20)、改良回数 ($K = 3$)、積のブロック化は無関係で、全列を補正すると FP64 の履歴と一致することから、補正する列数 $\max n < n$ （共線バンドは 2808 列中 604 列、非共線は 5616 列中 1036 列。クラスターは全列）が原因であった。占有ブロック内の回転を 1 次までにとどめる設計（同じ占有数の対は直交化のみ）のため占有ベクトルは FP32 程度の混合 v_{kj} を残し、補正しない列は占有部分空間に沿う FP32 程度の成分 d_{ki} を持ち続ける。このとき占有列 j の補正 E_{ij} の不動点は 0 ではなく $\sum_k d_{ki}(\lambda_k - \lambda_j)v_{kj}/(\lambda_j - \lambda_i)$ 、sidia333 で 10^{-8} 程度となり、密度の誤差 10^{-7} （NormRD の床）とウォームスタートでの偏りを生んでいた（補正後の列と補正しない列の直交性は FP32 程度の 4×10^{-6} ）。同じ占有数の対も回転させる案は近接した対の補正が 0.5 に達して直交性を壊し収束しなかった。採った対策は、補正しない列を毎ステップ補正済みの列に対して同じ式で回転させること（ $S_{rc} = Y^H X_c$ 、 $G_{rc} = X_r^H X_c$ 、 E_{rc} 、 $X_c \leftarrow X_c + X_r E_{rc}$ 。 $Y = AX_1$ を保持する 3 つ目のブロックを使う。積は約 6 割増、 $3n \max n \leq n^2$ ならメモリ増なし。OPENMX_EIGEN_REFINE_COMPLEMENT）で、共線バンド (10^{-12}) はコールド 24 回、ウォーム 23 回で FP64 (23 回) と 4×10^{-12} Ha 以内、NormRD の履歴は毎ステップ 3~4 桁一致し、直交性は 4×10^{-15} になった。ウォームスタートの偏りも消えたので、周期的な解き直しは不要になった（OPENMX_EIGEN_REFINE_WARM_PERIOD の既定を 0 に戻した）。RTX 5080 (18 ランク) でこの版 (10136f1e) を測り直すと、共線バンド (8 オーナー、直列ターン版 6+2) は 19 反復で FP64 と同じ、差 1×10^{-12} Ha、173 秒から 141 秒 (-19%)、スピン分極バンドは 22 反復 (FP64 は 23) で 220 秒から 173 秒 (-21%)、k 点を減らした 2 入力も反復増が消えて 146 秒から 118 秒、143 秒から 119 秒、非共線バンドは 23 反復で 322 秒から 224 秒 (-31%) となり、固有値計算の時間は各セットで 35~60% 減った。同時に、非共線バンドのウォームスタート用基底 (2k 点で約 1 GB) が SCF 後も GPU に残って力の計算がメモリ不足で落ちる不具合と、OpenMX の停止判定 (dUele のみ) が偶然通って共線クラスターが 5 反復早く止まる事象 (NormRD 4×10^{-8} で 2×10^{-9} Ha のずれ) が見付き、SCF サイクル終端での基底解放と、補正ステップでの停止に「`NormRD ≤ 100 × scf.criterion`」を課すガード (OPENMX_EIGEN_REFINE_STOP_GUARD) で対処した (9f75c70e)。RTX 5080 で 9f75c70e を測り直すと全 9 セット・20 本が正常終了し、非共線バンドは 23 反復で FP64 と同じエネルギー、325 秒から 213 秒 (-35%)、共線クラスターは停止ガードが 1 回働いて 20 反復で FP64 と一致した。全セットの短縮は非共線クラスター -24% 、バンド $-16 \sim -35\%$ 、共線クラスター $-2 \sim -3\%$ で、エネルギー差は最大 1.4×10^{-11} Ha、反復数は FP64 と同じか 1 回少ない。残る律速は RTX 5080 では Set_Hamiltonian (非共線クラスターで 190 秒中 89 秒) と Set_Density_Grid である。8 オーナーの実行でも補正の経路自体は動いている（各オーナーの FP32 解法と改良、部分占有クラスター 2 個のレイリー・リッツまで確認）。
- 電荷 -2 の系はフェルミ準位に部分占有の縮退状態を持ち、レイリー・リッツの対象になる。反復数は同じで、差は 10^{-11} Ha 以下である。最終 NormRD は FP64 と同じ桁 (4×10^{-9}) で止まる。

- 補正付きの解法には FP64 との一致を確かめる探針 (tests/eigen_precision_probe.cu) を用意し、改良回数、GEMM8 と倍精度の積、占有数の扱い、レイリー・リッツの有無、 δ の倍率を組み合わせで固有値・固有ベクトル・密度行列の誤差を測った。実装の前に、SCF から採取した $2n = 5616$ の行列で 2 回の改良が 10^{-12} の精度に達することを確認している。
- 実装中に見つけた問題は、2 回目の改良で δ を取り直すとクラスタが膨らむこと、占有状態だけを補正すると密度行列の精度が 10^{-10} まで落ちること (全 MaxN 状態の補正に戻した)、スピン分極の両所有者が同じ化学ポテンシャルを使う必要があること、cuBLAS を非同期ストリームに置くソルバーでは OpenACC のカーネルとの順序付けに同期が要ることである。

4.5 RTX 5080 での測定 (2026 年 10 月 6~7 日)

sidia333 を 18 ランク・MPS で測った (GeForce RTX 5080 16 GB、ドライバ 595.91.07、CUDA 13.4)。D = 既定 (補正あり。GeForce では自動で有効になる)、R0 = OPENMX_EIGEN_REFINE=0 (FP64 の固有値解法、ほかは同じ)、Dw0 = ウォームスタートなし、Dt0 = 二段階 FP32 解法なし、旧 = v1.3 時点の既定である。最終版 (コミット 1f424114) の全計算の時間と、途中の版の値を示す。

コミット・設定	中性		scf.system.charge= -2	
	反復	全計算 (s)	反復	全計算 (s)
旧既定 (v1.3、FP64 の固有値解法、密度行列は状態和)	23	261.3	21	236.1
88f1b8dd D (補正のみ)	23	218.7	21	194.6
1009bf80 D (補正+密度行列の積化)	23	197.9	21	185.9
1f424114 R0 (FP64、密度行列の積化)	23	249.8	21	232.4
1f424114 Dw0 (ウォームスタートなし)	21	179.7	—	—
1f424114 Dt0 (一段階 FP32 解法)	23	186.9	—	—
1f424114 D (最終版)	21	173.9	21	173.7

密行列解法 1 回あたりの段階の時間 (ms、2 回目以降の中央値) と、全計算の内訳 (s) は次のとおりである。

1f424114、固有値解法	密度行列	順変換	密行列解法の合計	Set_Hamiltonian	Set_Density_Grid	
中性						
R0	3,189	135	56	3,421	86.5	35.5
Dw0	490	133	56	720	80.9	32.8
Dt0	163	134	56	395	85.3	35.6
D	163	133	56	395	79.4	32.9

- 精度：全エネルギーの差は FP64 に対して中性で 7×10^{-12} Ha、電荷 -2 で 1×10^{-11} Ha、力の最大差は 2×10^{-12} (中性) と 2×10^{-11} Ha/bohr (電荷 -2)、化学ポテンシャルとスピンモーメントの差は 10^{-12} 以下である。最終 NormRD は FP64 と同じ桁で止まった。D は中性で反復が 2 回少なく (21 回)、収束先は同じである。
- 速度：固有値解法の段階は 3,189 ms から 163 ms (ウォームスタートの採用は 21 回中 19 回、拒否は初期の 5 回)、密行列解法 1 回は 3.42 秒から 0.40 秒になった。全計算は 30 % 短い。Dw0 との差 (490 ms 対 163 ms) がウォームスタートの効果、Dt0 との差が二段階 FP32 解法の効果である (一段階では FP32 解法が 1 回 900 ms、二段階では 550 ms)。
- 密度行列の積化 (1009bf80) は、状態ごとの和の 427 ms を 133 ms にした。全計算では約 20 秒である。
- 共線クラスター ($n = 2808$)：スピン Off で固有値解法が 187 ms から 38 ms、全計算は 94.6 秒から 92.2 秒 (20 反復、差 0)。スピン On で 326 ms から 66 ms、116.6 秒から 111.5 秒 (差 8×10^{-12} Ha)。この大きさでは固有値解法は全計算の数 % で、残りは格子上の処理 (Set_Hamiltonian30~43 秒、Set_Density_Grid23 秒、力 11 秒) である。
- D の内訳 (173.9 秒) は、Set_Hamiltonian が 79.4 秒 (1 ステップ 3.8 秒、46 %)、Set_Density_Grid が 32.9 秒 (19 %)、対角化が 19.8 秒 (11 %)。うち密行列解法が 8.3 秒)、力が 12.1 秒、Set_ProExpn_VNA が 6.0 秒、混合が 4.6 秒である。固有値解法はもう最大の項ではなく、格子上の FP64 の処理が全体の 3 分の 2 を占める。
- 共線バンド・非共線バンドの RTX 5080 での測定は、A100 の検証を終えてから手順書に加える。

4.6 制限と次の対象

- 1 ランクに複数の k 点を持つバンドの経路 (all_knum $\neq$ 1) にも実装した。1 回目のパスでは各 k 点を FP32 で解いて前のステップの化学ポテンシャルで補正し (固有値は固有ベクトルの誤差の 2 次なのでこれで足りる)、基底をホストに保持する (ランクあたり OPENMX_BAND_REFINE_BASIS_MB、既定 1024 MB)。2 回目のパスではその基底からこのステップの化学ポテンシャルで補正して固有ベクトルを作る。A100 の検証 (共線 Off、4 ランクで 2k 点/ランク：FP64 と 9×10^{-12} Ha；非共線、2 ランクで 2k 点/ランク： 2×10^{-11} Ha) で一致した。1 ランク 1k 点の経路で全オーナーのバッファが同時に GPU に載らない (ターンが直列化される) 場合も、複数 k 点経路と同じ 2 パス (1 パス目は前ステップの化学ポテンシャルで補正して基底をホストへ退避、全 k 点の推定値から化学ポテンシャルを決め、2 パス目はその基底から補正) に分けて補正するようにした。RTX 5080 (16 GB) では 8 オーナーのバンド入力で 6~7 オーナーしか 1 ターン群に入らず補正が使われていなかったが、A100 でターン群を 4 に制限して模擬すると (共線バンド、8 オーナー、 10^{-10})、FP64 (直列) 19 反復 -887.422830856532 Ha に対し補正版は 21 反復 -887.422830856499 Ha (差 3.3×10^{-11}) で動作した。
- 補正の積は GEMM8 の精度に依存する。積が cuBLAS の FP64 に退避した場合、GeForce では補正の利点が消えるので実行全体を FP64 に戻す。

- RTX 5080 で残る時間の大半は Set_Hamiltonian と Set_Density_Grid である。ただしその原因は FP64 演算の遅さではない。Set_Hamiltonian の GPU 経路は軌道の格子値などの表（1 ランクあたり約 2.4 GiB、A100 では 8 ランクで 18.9 GiB）を GPU に常駐させる方式で、16 GB の RTX 5080 では 18 ランクのうち 4~7 ランクしか GPU に入らず、残りは CPU で計算している（ログの CPU fallback ranks=11~14）。Set_Density_Grid の分散 GPU 積分はこの常駐表を前提にするため、RTX 5080 では GPU を使えていない。A100（8 ランク、すべて GPU）では両者が 1 ステップ 1.4 秒 + 1.0 秒、RTX 5080 では 3.8 秒 + 1.5 秒である。次の対象はこの二つの格子処理のメモリ設計（直列ウェーブ、ストリーミング、軌道値の GPU 上での再計算）と、対角化のうち密行列解法以外の 0.4~0.7 秒/ステップである。

5 格子上の処理の GPU 化：軌道値のその場計算と double-float

5.1 RTX 5080 での時間の内訳と原因

run4（コミット 9f75c70e）の非共線クラスター D の 190 秒は、Set_Hamiltonian が 89 秒、Set_Density_Grid が 36 秒、対角化が 22 秒、力が 12 秒である。Set_Hamiltonian のステップごとの内訳（SETHPROF）を見ると、表を GPU に常駐できた 4~7 ランクは行列要素の計算に 0.5~0.7 秒、常駐できず CPU で計算する 11~14 ランクは 3.1~3.6 秒かかり、全ランクがその終了を待っている。軌道の格子値の表（orbs1+orbs0+nolg）は 1 ランク 1.24 GiB、18 ランクで 22 GiB あり、16 GB のカードには入らない。GPU 側のカーネル（原子対ごとに格子点の和を FP64 で累積）は RTX 5080 では FP64 の FMA 性能（FP32 の 1/64）で律速され、1 ステップ約 1.2×10^{12} 回の FP64 FMA に 2.7 秒程度を要する。すなわち、(1) 全ランクを GPU に載せること、(2) FP64 演算量を減らすこと、の 2 つが必要である。

5.2 却下した案：SCF 序盤の FP32 化

固有値解法と同じ発想で、SCF 序盤は FP32（補償付き和）で格子積分し、NormRD が閾値を切ったら FP64 に切り替える段階的精度を実装して試した（OPENMX_GRID_FP32、閾値 OPENMX_GRID_FP32_UNTIL、切替時に混合履歴を再開）。A100 の非共線クラスター（8 ランク、 10^{-10} ）では、FP64 の 23 反復に対し、閾値 10^{-4} で 27、 10^{-5} で 34、 10^{-6} で 40 反復になった。FP32 のステップはカーネルの誤差水準（NormRD で $10^{-4} \sim 10^{-5}$ ）で SCF が停滞し、節約できる以上に反復が増える。固有値解法では「解を補正で真値に戻す」ので FP32 が使えたが、格子積分では FP32 の誤差がそのまま密度に入るため、この方式は不適と判断した（実装は実験用に残し既定はオフ）。

5.3 軌道値のその場計算（表を持たない）

Set_Orbitals_Grid.c には表を作るための軌道値の GPU 計算（ホストの Get_Orbitals の忠実な転記、FP64 で計算して float で格納）がすでにある。これを共用部品（orbs_grid_gpu.h：種ごとの PAO 動径表・セル並進・格子の枠をデバイスに常駐させ、原子対のバッチの重なり格子点で軌道値を評価する）にし、表を常駐できないランクは Set_Hamiltonian の行列要素を原子対のバッチ（作業領域 256 MiB）ごと

に「軌道値の評価→既存のカーネル」で計算するようにした (OPENMX_SETHAM_ONTHEFLY、既定オン)。デバイスに残るのは自分の原子の球内格子点の索引 (数 MiB) だけで、転送も表のホスト詰め替えも要らない。他ランクの原子の軌道値は表を作るのと同じ計算なのでビット単位で一致し、自ランクの近傍原子については格子枠の取り方 (中心原子の枠か近傍原子の枠か) が違うため最下位ビットで一部異なる。Set_Density_Grid の分散 GPU 積分も、常駐表のないランクでは同じタイルから対ごとに密度を足し込む (原子的加算) 方式にし、常駐ランクと混在できるようにした。A100 で RTX 5080 のメモリ上限を模擬した結果 (非共線クラスター、8 ランクのうち 3 ランク常駐、12 反復の Set_Hamiltonian) は、全常駐 17.3 秒、その場計算 19.0 秒 (1 ランク 1 ステップ: 詰め替え 0.17 秒+評価 0.26 秒+カーネル 0.41 秒)、従来の CPU 退避 165 秒で、固有値和は 12 反復目で常駐版と 2×10^{-12} Ha 以内に一致した (途中の反復では最下位ビットの差が SCF の軌跡で増幅され最大 4×10^{-8} Ha ずれるが、収束先は同じ)。

5.4 double-float カーネル

FP64 演算を減らすため、行列要素と密度のカーネルに double-float 版を入れた (OPENMX_GRID_PRECISION=df。FP32:FP64 の性能比が 8 以上の GPU で既定)。重み付き軌道値や密度行列を hi/lo の 2 つの float に分け、積は FMA で誤差なしに作り (TwoProd)、タイル内の和は Neumaier の補償付き加算、タイル間の累積は FP64 で行う。1 項あたり FP32 命令約 12 個で、RTX 5080 では FP64 FMA 1 個 (FP32 の 64 命令分) より約 5 倍速い計算量になる。誤差はタイルあたり $32 \epsilon_{32}^2 \approx 4 \times 10^{-13}$ の相対誤差で、FP64 カーネルと同程度の結果を期待している (A100 での精度検証と RTX 5080 での時間測定は本稿執筆時点で進行中)。

5.5 RTX 5080 での測定 (run5、2026 年 10 月 8 日)

非共線クラスター D (18 ランク、23 反復) は、run4 の 189.8 秒 (Set_Hamiltonian89.0 秒、Set_Density_Grid35.6 秒) から、その場計算+FP64 カーネルで 159.3 秒 (54.1 秒、39.3 秒)、その場計算+double-float で 139.5 秒 (42.8 秒、31.3 秒) になり、4 本のエネルギーは完全に一致した。他のセットも D は R0 より 15~30% 短く、エネルギー差は最大 3×10^{-11} Ha である。1 ランク 1 ステップの内訳は詰め替え 0.19 秒、軌道値の評価 0.70 秒、カーネル 0.50 秒で、評価が最大の項目になった。同時に、(1) 作業領域の既定 256 MiB×11 ランクが常駐ランクの分と合わせて 16 GB を超え常駐ランクのカーネルが落ちる (128 MiB なら入る)、(2) 非共線バンドで SCF 後の力の計算がメモリ不足で落ちる (その場計算の索引や密度の常駐バッファが数百 MB 残る)、の 2 点が見つかり、作業領域を「空き-常駐ランクの必要量-余裕 256 MiB」の分け前で自動決定すること、SCF サイクル終端でその場計算の状態と密度の常駐バッファを解放すること、評価から三角関数と二分探索を除くこと (値は double の最下位ビットまで同じ。A100 で表が同一のエネルギーを与えることを確認) で対処した (29fla7ca。RTX 5080 での再測定待ち)。

5.6 RTX 5080 での再測定 (run6) と開発の移管

修正後 (29f1a7ca) の全 9 セットは判定基準を満たし、非共線バンドも力の計算まで完走した (R0 320.3 秒 → D 212.1 秒)。一方、作業領域の自動予算は 16 GB のカードでは小さすぎた (54~63 MiB)。2 ステップ目から常駐ランクが 7 から 4 に減り (1 ステップ目にその場計算のランクが確保した索引と密度のバッファの分だけ空きが減るため)、余裕 256 MiB を引くと 14 の非常駐ランクのうち 9~11 しか作業領域を得られず、残りは CPU に落ちた。非共線クラスター D は既定で 197.3 秒、余裕 0 で 165.0 秒、上限 128 MiB と余裕 0 で 149.1 秒 (run5 の 139.5 秒には届かない)。密度側の作業領域も既定 256 MiB では毎ステップ 1~2 ランクが GPU 積分を失い、128 MiB なら全ランクが GPU で積分する (41~53 秒 → 31.8 秒)。三角関数と二分探索を除いた評価は値を変えずに同じ時間だった (1 ランク 1 ステップ 0.6~0.7 秒)。評価する値の数あたりの速さが A100 と RTX で等しいことから、評価は FP64 演算律速ではなく、占有率・ローカルメモリ・転送か、MPS 下の時分割によるものと推定される。以後の開発は RTX 5080 側で行うこととし (2026 年 10 月 8 日)、予算と計画の安定化、密度側の予算、Nsight による評価カーネルの分析を優先課題として引き継いだ (引き継ぎ文書 HANDOVER_RTX5080_20261008.md)。

5.7 RTX 5080 側での修正 (2026 年 10 月 8 日、コミット a55baab4)

開発を RTX 5080 側へ移した後、run6 で残った「既定値のままでは遅い」問題を次の三点で直した。(1) その場計算の予算を、ステージング組が確保中に測った `cudaMemGetInfo` の値ではなく計画値 (計画時の空き - ステージング組のピーク) だけから決める。測った値はランクごと・ステップごとに揺れ、作業領域がほぼ毎ステップ変わり (バナー 23 回中 21 回)、256 MiB の余裕のもとでは 14 ランク中 3 ランクが領域を得られず CPU に落ちていた。余裕 `OPENMX_SETHAM_ONTHEFLY_MARGIN_MB` の既定を 0、上限 `OPENMX_SETHAM_ONTHEFLY_MB` の既定を 128 MiB とし、分け前が 48 MiB を下回っても 48 MiB を与える (CPU に落ちるのは確保に失敗したときだけ)。(2) 相の時間を決めるのはステージング組ではなくその場計算のランクである (run5b のランク別計時: ステージング組は 1 ステップ 0.6~0.7 秒の後にバリアで 0.8 秒待ち、その場計算のランクは 1.4 秒。ステージング組を 3・2・1 ランクに制限する実験でも `Set_Hamiltonian` は 43~45 秒で変わらない)。そこで行列要素の計画は、波の外の各ランクが上限分の作業領域を得られるまでステージング組を縮める (全ランクが 1 波に載る GPU では発動しない)。波は 1 ステップ目 6 ランク、2 ステップ目以降 3 ランクで固定され、作業領域は 128 MiB に保たれる。(3) 密度のその場計算は、計画の非常駐数ではなく実際にその場計算を取るランク数 (常駐テーブルのない全ランク = ハイブリッド時は 18 ランク) で予算を割り、専用の上限 `OPENMX_SETDEN_ONTHEFLY_MB` (既定 128 MiB) を持つ。256 MiB では毎ステップ 1~2 ランクが GPU 積分を失っていた。

結果 (18 ランク、MPS、環境変数なし、エネルギーはすべて FP64 と 10^{-12} Ha で一致): 非共線クラスター D は 197.3 秒から 141.4 秒 (`Set_Hamiltonian` 78.1 → 43.0 秒、`Set_Density_Grid` 53.1 → 32.5 秒。run5b の 128 MiB 指定時 139.5 秒と同等)、非共線バンド D は 210.6 秒から 167.5 秒 (`Set_Hamiltonian` 85.9 → 42.9 秒)。その場計算のランクは毎ステップ全員が GPU に載り (SETHOTFPROF 15/15、SETHOTFDENPROF 18/18)、バナーは 1run に 2 回である。なお非共線バンドの R0 (cuSOLVER の

FP64 エミュレーション) が 18 ステップ目に `cusolverDnXsyevdx` の非収束 (`info=126`) で 1 回落ちたが再実行では完走しており、散発的な非収束と判断した。固有値解法の呼び出しは `info≠0` で中止するだけなので、ネイティブ FP64 で解き直す再試行 (解く前のデバイス上の行列の退避が要る) を候補とする。

5.8 軌道値評価の double-float 化 (2026 年 10 月 8 日)

RTX 5080 側でのプロファイル (1 ランク、Nsight Systems) では、その場計算の軌道値評価カーネル (`SOG_Device_EvalPairTiles`) が全 GPU 時間の 29 % を占め、18 ランクではランク・ステップあたり 0.75 秒 (ハミルトニアン) と 0.42 秒 (密度) であった。評価点数 (1 ステップ約 4×10^8 点) と 1 点あたりの FP64 命令数 (スプライン・対数・平方根・除算で約 800) から見積もると、GeForce の FP64 パイプ (FP32 の 1/64、RTX 5080 で約 0.9 TFLOPS) がちょうど飽和する時間と一致する。つまり `Get_Orbitals` の忠実な FP64 転記は、GeForce では FP64 演算そのものに律速されている。

そこで評価を double-float (float の対 hi+lo、相対誤差 $\sim 2^{-48}$) に書き換えた (`OPENMX_ORBS_EVAL_PRECISION`、既定 `df`、`fp64` で従来の経路)。FP64 のまま残すのは格子座標 (大きな座標の差で桁落ちするため) とメッシュ添字の比較だけで、PAO の動径テーブルとメッシュ点ごとのスプライン定数 (h_2 、`dum1...4`) は対として事前にデバイスに置き、メッシュ添字は FP32 の対数で見積もって添字探索で確定する。書き出す float は積の hi 部であり、FP64 値が float の丸め境界の 10^{-14} 以内にある場合 (約 10^6 個に 1 個) だけ 1 ulp 異なる。テーブル (常駐ランクの `Orbs_Grid`) は従来どおり FP64 で作る。

結果 (18 ランク、MPS、環境変数なし) : 非共線クラスターでは評価がランク・ステップあたり 0.77 秒 $\rightarrow$ 0.41 秒 (ハミルトニアン)、0.42 秒 $\rightarrow$ 0.07 秒 (密度)、`Set_Hamiltonian` が 43.5 秒 $\rightarrow$ 35.8 秒、`Set_Density_Grid` が 32.6 秒 $\rightarrow$ 24.1 秒、D が 141.4 秒 $\rightarrow$ 129.5 秒で、`Utot` は FP64 評価 (`R0`、`fp64` 指定) と 10^{-12} Ha まで同一 (1 ランク 3 ステップでも 10^{-12} Ha)。非共線バンドの D は 167.5 秒 $\rightarrow$ 156.8 秒 (`Utot` 差 10^{-12} Ha)。ハミルトニアンの評価が密度ほど縮まないのは、同時に走る他ランクの行列要素カーネル (次項) が FP64 パイプを占有しているためである。

5.9 格子積分カーネルからの FP64 の除去

double-float の行列要素カーネル (`matrix_elements_kernel<2>`) は、各タイルの各点・各軌道について $V(\mathbf{r}) \phi_i(\mathbf{r}) dV$ を FP64 で掛けてから hi/lo に分けており、しかもこれを出力ブロック (256 スレッド、非共線では 3 ブロック) ごとに繰り返していた。1 点あたり FP64 命令約 470 で、GeForce では 1 ステップ 0.45 秒に相当し、観測された「device」0.5 秒の大半を説明する。密度カーネル (`Set_Hamiltonian_OTF_DensityKernel`) も行の和 $f_0 t$ を FP64 で累積していた (1 点あたり 156 命令、0.15 秒)。前者はタイルの点ごとに $V dV$ を一度だけ hi/lo に分け (カーネル内で唯一の FP64)、軌道との積は FMA で float の対として作る形に、後者は行の和を float の対 (TwoSum) で累積して原子的加算のときだけ FP64 にする形に改め、両者とも lo 側の積は補償なしの単純加算にした (lo は hi の 2^{-24} 倍で、タイルあたりの誤差は従来と同じ $32 \epsilon_{32}^2$ の次数)。結果 (18 ランク、MPS、環境変数なし) : 非共線クラスターでは、その場計算ランクのステップあたりの行列要素が 1.42 秒 $\rightarrow$ 0.61 秒 (評価 0.41 $\rightarrow$ 0.15 秒、積分 0.52 $\rightarrow$ 0.23 秒)、`Set_Hamiltonian` が 35.8 秒 $\rightarrow$ 24.9 秒、`Set_Density_Grid` が 24.1 秒 $\rightarrow$ 22.0 秒

となり、D は 129.5 秒 → **116.3 秒** (run4 の 189.8 秒の 61%)、R0 は 192.2 秒 → 183.4 秒。FP64 カーネル (OPENMX_GRID_PRECISION=fp64) の R0f64 208.2 秒・Df64 145.7 秒と比べた Utot の差は 10^{-12} Ha である。非共線バンドは R0 266.6 秒 → 254.3 秒、D 156.8 秒 → **144.4 秒** (Utot 差 2×10^{-12} Ha)。残るのは各ランク共通の Set_Vpot (CPU の交換相関、0.36 秒/ステップ)、密度カーネル (0.37 秒)、対の束の詰め替え (0.23 秒) である。

5.10 格子上的交換相関ポテンシャルの GPU 化

行列要素の後に残った共通費用のうち最大のものは Set_Vpot の 0.36 秒/ステップで、その中身は Set_XC_Grid (GGA-PBE: 27 点ステンシルによる密度勾配、点ごとの汎関数、発散、非共線の回転) のランクごとに単一スレッドで走るホストループであった。これを 4 つの OpenACC カーネル (XC_PBE・XC_PW92C・XC_EX のデバイス転記を含む) として FP64 のままデバイスに移した (OPENMX_XC_GRID_GPU、既定 1。LDA や LDA+U の cFLL 回転などはホストに戻る)。ランクの D 格子は 10^5 点程度で FP64 でも数ミリ秒であり、18 ランクでは Set_Vpot が 0.35 秒 → 0.18 秒/ステップ (残りは格子の再分配)、Set_Hamiltonian が 24.9 秒 → 20.2 秒、非共線クラスターの D は 117.1 秒 → **111.1 秒** (R0 181.8 秒 → 173.7 秒) となり、Utot はホスト計算と 10^{-12} Ha で一致した (1 ランクで全格子を扱う場合は 2.38 秒 → 0.21 秒/ステップ)。

5.11 FP64 エミュレーション参照解法の再試行

GeForce での FP64 参照 (R0) は cuSOLVER の固定小数点 FP64 エミュレーションで解いているが、この解法はまれに収束しない (RTX 5080 で 2026 年 10 月 8 日に約 700 回中 2 回: バンド Col で info=395、バンド NonCol で info=126)。従来は呼び出し側が info $\neq$ 0 で中止していたため、測定の途中で R0 が落ちることがあった。cusolverDnXsyevevdx/Xsyevevdx の互換ラッパーで、エミュレーションハンドルの FP64 解法の前行列の写し (デバイス、足りなければホスト) を取り、info>0 なら行列を戻してネイティブ FP64 で解き直し、ハンドルをエミュレーションに戻すようにした (作業領域は両モードの大きい方を確保)。失敗は散発的で、再試行の費用は失敗時のネイティブ解 1 回分と、毎回の写し (デバイス上では 1ms 未満) だけである。

5.12 DC・Krylov・DC-LNO への密度 GPU 経路の適用 (2026 年 10 月 9 日)

Set_Hamiltonian のその場計算・double-float カーネル・GPU 交換相関はソルバに依らず効くが、Set_Density_Grid の GPU 経路 (分散求積と単一ランクサービス) はクラスター・バンドソルバに限定されていた。どのソルバも Set_Density_Grid に FNAN 近接までの同じ密度行列を渡すので、分散求積 (各ランクが自分の原子を、常駐テーブルかその場計算のタイルで積分) の条件を DC・Krylov・DC-LNO にも広げた。単一ランクサービスは全格子のテーブルを 1 台のデバイスに作るため $O(N)$ ソルバの系では収まらず、クラスター・バンドのままとした。RTX 5080 (18 ランク、MPS) の runtest の deck で、Set_Density_Grid は DIA512-1 (Krylov、512 原子) 8.42 秒 → 6.02 秒、MCCN (Krylov) 14.55 秒 → 8.75 秒、CG15c-DC-LNO 10.35 秒 → 7.41 秒、非共線 DC-LNO (216 原子、3 ステップ) 13.73 秒 → 7.39 秒

で、 U_{tot} は CPU 密度の計算と $10^{-12} \sim 3 \times 10^{-11}$ Ha で一致した (C60 の DC では 1.2×10^{-10} Ha : 1 ulp の密度差を rmm-diisk 混合が増幅する既知の性質)。

5.13 密度カーネルのレジスタタイリングと Set_Vpot の残り (2026 年 10 月 9 日)

その場計算の密度カーネルは、点ごとに密度行列を毎積でグローバルメモリから読み、軌道行を動的添字で参照していたため、格子処理に残る最大のカーネル (ランク・ステップあたり 0.37 秒) だった。これを CUDA カーネルに書き換え、対の密度行列 (hi/lo の float 対) をブロックごとに一度だけ共有メモリへ置き、点の軌道行 $\phi_1 \cdot \phi_0$ をレジスタに保持する (軌道数の上限 16/32/48 の 3 段をコンパイル時定数とし、ループを展開して添字を静的にする)。演算は従来と同一 (FMA による正確な積、hi 側の Neumaier 補償、lo 側の単純加算、行の和は float 対、点・スピンごとに 1 回の FP64 原子的加算)。18 ランクで 0.37 秒 $\rightarrow$ 0.098 秒 (1 ランクでは 3.55 秒 $\rightarrow$ 0.23 秒)、 $Set_Density_Grid$ は 21.9 秒 $\rightarrow$ 17.8 秒、非共線クラスターの D は 111.1 秒 $\rightarrow$ **107.7 秒 (3 回の実行で 105.8~107.7 秒)** (U_{tot} 差 10^{-12} Ha)。

Set_Vpot には段階別の計時 (SETVPOTPROF) を入れた。18 ランクでの内訳は XC 0.05 秒、 V_{xc} の $D \rightarrow B$ コピー 0.002 秒、組み立て 0.002 秒、 $B \rightarrow C$ 再分配 0.14 秒であり、残りは実質的にポテンシャルの $B \rightarrow C$ 再分配 (ランクあたり約 32 MiB の MPI 転送と詰め替え) である。毎ステップの malloc/free をやめて作業配列を保持することで 0.14 秒 $\rightarrow$ 0.11 秒になった。XC の 0.05 秒は 18 ランクが同時にカーネルを投入する際のデバイス占有の順番待ちで、転送を 1 本の非同期キューにまとめることもアリーナの常駐化も効果がなく採用しなかった。 $Set_Density_Grid$ にも同様の計時 (SETDENPROF) を入れた: 判定 0.03 秒、求積 0.43 秒 (詰め替え 0.16、評価 0.17、カーネル 0.08)、 $A \rightarrow B$ 通信 0.08 秒、重ね合わせ 0.05 秒、 $B \rightarrow D$ 0.04 秒。次の候補は、行列要素と密度に共通する「詰め替え」(ステップ間で不変なバッチ構造を毎ステップ組み直し、小さな転送を数百回行っている) のキャッシュ化で、合計 0.3~0.4 秒/ステップの短縮が見込める。

5.14 詰め替えのキャッシュ化 (2026 年 10 月 9 日)

その場計算の両経路 (行列要素・密度) は、バッチ (どの対か・各オフセット)、対の記述子、点ごとの索引 (N_c 、行列要素ではポテンシャルの格子添字 MN) を SCF ステップごとに組み直し、バッチあたり約 90 回の小さな同期転送 (N_c リストは対ごと) で送っていた。18 ランクの MPS 下ではこの転送 1 回が約 0.3 ms かかり、詰め替えが行列要素で 0.25 秒、密度で 0.20 秒 (ランク・ステップあたり) を占めていた。これらは SCF ステップに依存しないので、ランクごとに「計画」として保持し、各バッチの静的な部分をデバイスのアリーナと同じ配置のピン留めホスト画像にまとめて 1 回で転送するようにした (OPENMX_SETHAM_ONTHEFLY_PLAN_CACHE、既定 1)。毎ステップ動くのは H ブロック (ピン留めバッファ)、ポテンシャル行 (cudaHostRegister で一度だけ登録)、タイルだけで、密度の累積配列はデバイス側でゼロ化し (以前は毎ステップ約 50 MB のゼロを転送していた)、そのホストバッファもピン留めして持ち越す。結果 (18 ランク、MPS): 行列要素の詰め替え 0.25 秒 $\rightarrow$ 0.10 秒、密度の詰め替え 0.20 秒 $\rightarrow$ 0.02 秒、その場計算ランクの行列要素 0.68 秒 $\rightarrow$ 0.48 秒、密度 0.41 秒 $\rightarrow$ 0.26 秒、 $Set_Density_Grid$ 18.1 秒 $\rightarrow$ 14.8 秒、非共線クラスターの D は 107.7 秒 $\rightarrow$ **103.2 秒**、非共線バンドの D は 144.4 秒 $\rightarrow$ 133.8 秒 (U_{tot} 差

10^{-12} Ha、runtest の 14 deck も変化なし)。行列要素の段階は、残る 0.48 秒のうち積分カーネル待ちが 0.24 秒で、15 ランクのカーネルがデバイスを分け合う総時間に律速されている。次に効くのは行列要素カーネル自体（レジスタタイリング）である。

5.15 見込み

RTX 5080 では、全ランクが GPU に載ることで Set_Hamiltonian の 1 ステップが CPU 退避の 3.6 秒から「評価＋カーネル」に置き換わり、double-float でカーネルがさらに縮むので、89 秒が 20～30 秒程度、Set_Density_Grid の 36 秒も同程度に縮むと見込む。残る未知数は GeForce での軌道値評価（FP64 の三角関数・スプライン）の速さで、A100 では演算律速ではなくメモリ遅延律速だった。

6 数学的対象と再利用の成立条件

6.1 固定された直交化行列

SCF 反復番号を t とし、基底 n 、保持する部分空間の次元 $r \leq n$ について、

$$H_t C_t = S C_t \varepsilon_t, \quad X^\dagger S X = I_r, \quad X \in \mathbb{C}^{n \times r} \quad (1)$$

を考える。標準固有値問題への変換と逆変換は

$$\tilde{H}_t = X^\dagger H_t X, \quad \tilde{H}_t Y_t = Y_t \varepsilon_t, \quad C_t = X Y_t. \quad (2)$$

最初の対象は

$$B_t = H_t X \in \mathbb{C}^{n \times r}, \quad \tilde{H}_t = X^\dagger B_t \in \mathbb{C}^{r \times r} \quad (3)$$

の二演算である。固有値ソルバーと $C_t = X Y_t$ は対象外とし、現行の経路のまま実行する。

監査で確認した X の扱いは経路ごとに異なる。

- **共線クラスター**：実正方行列 ($r = n$)。構造ごとに一度作られ、SCF の間は同じデバイス領域に残る。実装はここから始めた。スピン分極では、スピンのごとに別のランクが同じ X で解く。
- **非共線クラスター**：同じ実行列 X で六つの実ブロックを変換する（1 反復あたり 12 個の GEMM）。未着手。
- **複数 $\mathbf{k}$ 点のバンド計算**： $X(\mathbf{k})$ はホスト側のキャッシュにあり、 $\mathbf{k}$ 点ごとに同じデバイス領域へ転送される。一つのポインタが異なる $X(\mathbf{k})$ を指すので、再利用には $\mathbf{k}$ 点ごとの保持領域が要る。未着手。

このため X はポインタではなく、識別番号と更新番号で区別する。 X を作り直すたびに更新番号が増え、古い準備済み表現は使われない。

6.2 不変な行列と不変な剰余表現を区別する

moduli 集合を $\{m_\ell\}_{\ell=1}^L$ とすると、CRT の復元範囲は $M_L = \prod_{\ell=1}^L m_\ell$ に依存し、スケーリングは整数化した積をこの範囲に収めるために使われる [2]。moduli 数やスケーリング方式を変えれば、準備済み表

現は作り直す。これは実装でも確認した。準備済み表現には（更新番号、形状、op、左右の役割、moduli 数、スケーリング方式）を付け、一つでも違えば前処理をやり直す。

v1.2 は「準備処理が固定行列だけから決まるのか、相手行列にも依存するのか」を監査項目とした。答えはスケーリング方式による（第 2 節）。

- fast mode では固定行列だけで決まり、再利用は厳密である。
- accurate mode では相手行列に依存する。再利用すると結果が変わり、精度が約 1 modulus 分落ちる。

従って **fast mode** を主設定とし、accurate mode では再利用を行わない。accurate mode で再利用する方式は、別の数値方式として扱う。なお、測定した五つの系では、fast mode の誤差は accurate mode と同等かそれより小さかった。

再利用単位 構造・値・演算条件・精度設定が同じ区間で、役割（ HX の右側、 $X^\dagger B$ の左側）ごとに一つの準備済み表現を再利用する。実測では、60 反復の GFRAG で前処理 2 回・再利用 118 回、3 段階の精度制御では前処理 6 回であった。スピン分極の場合はスピンを担当するランクごとに同じ回数になる。

6.3 変換誤差と悪条件性

計算された二つの積を $\hat{B}_t = H_t X + E_{1,t}$ 、 $\hat{H}_t = X^\dagger \hat{B}_t + E_{2,t}$ と書くと、

$$\Delta_t = \hat{H}_t - X^\dagger H_t X = X^\dagger E_{1,t} + E_{2,t}, \quad \|\Delta_t\|_F \leq \|X\|_2 \|E_{1,t}\|_F + \|E_{2,t}\|_F. \quad (4)$$

保持した最小固有値が小さいと $\|X\|_2 = 1/\sqrt{s_{\min}}$ が大きくなるため、 S の固有値の範囲を記録する（プロープが出力する）。

v1.2 は、誤差監視の対象を「ソルバーが読む三角部分から作ったエルミート行列」とした。実測では、これは使えなかった。 H_t 自体が 4×10^{-10} 程度の非対称性を持つため、三角部分から作った行列と参照値の差は、積の精度によらず $3 \times 10^{-10} \sim 9 \times 10^{-10}$ で頭打ちになる（cuBLAS FP64 でも同じ）。そこで誤差監視は計算されたままの全行列 $\hat{H}_t$ を対象とする。ソルバーが読む行列との違いは H_t の非対称性に由来するもので、A~F のすべての条件に共通である。この非対称性そのものの由来と影響は、精度制御とは別の課題として記録する。

7 実装の現状と GEMMu8 への API 追加案

7.1 OpenMX 側に実装したもの

ブランチ adaptive-precision に次を実装した。いずれも共線クラスター経路が対象である。

機能	内容
順変換専用の入口	二つの GEMM だけが通る入口。精度段階（既定・FP64・GEMMu8）を他の GEMM と独立に選ぶ
固定設定での再利用	既存の <code>skip_scalA/B</code> を使い、準備済みの X を専用の領域に保持する。識別番号と更新番号で管理し、回数・時間・保持量を数える
全量確保の既定化	第 7.2 節
精度制御	段階列、残差による昇格、試行の棄却と再計算、最終段階の規則（第 8 節）。入力キーワード <code>scf.gemmul8.adaptive.*</code>
誤差指標	乱数方向による検査（第 8.2 節）
測定の道具	行列採取、行列単体のプローブ（倍々精度の参照、ビット比較、内部計時、容量表）、密行列解法の段階別計時、設定を変えた実行の一括比較
回帰試験	260 項目。実数・複素数、長方形、転置・共役転置、左右の役割、値の更新、複数の X の交互利用、moduli 数の変更、容量不足時の切替、上限の前後、精度制御の状態遷移

v1.2 の回帰項目のうち、複素数 k 点、ストリーム間の依存関係、準備途中の失敗は試験していない。

7.2 既定動作の変更：メモリが足りるときは全量確保

順変換の二つの GEMM は、既定の精度設定のまま、デバイスに空きがあれば作業領域を全量確保して実行するように変えた。

- 空きが「必要量＋予備 1.5 GiB」に満たなければ、従来の分割実行に戻る。精度は変えない。
- 作業領域は順変換の直後に返し、固有値解法と固有ベクトルの保持が従来と同じ空きを見るようにした。
- 256 MiB 以下で足りる大きさでは、従来の経路とビット単位で同じ結果になる（GFRAG、Pt63、Mn12_148_F、DIA512-1 で確認）。
- `OPENMX_GEMMUL8_FORWARD_UNBLOCKED=0` で従来の動作に戻せる。

効果は第 2 節の表の「旧既定」と「新既定」の差である。順変換は CG15c で 106 ms から 56 ms、MCCN' で 145 ms から 76 ms になった。 $n = 13,000$ (c1000) では 810 ms から 421 ms になり、密行列解法に占める割合は 17% から 10% に下がった。24 ランクが MPS で共有する 40 GB の A100 で、容量による切替は起きなかった。順変換以外の GEMM（逆変換など）は従来どおり上限つきで動く。全ランクが作業領域を持つ経路に同じ変更を広げるかどうかは、ランク数と GPU メモリ関係を見て別に判断する。

7.3 GEMMu8 への明示的な API：提案しない

v1.2 が提案した概念 API を再掲する。既存の関数名ではない。

```
# Once per valid geometry / precision interval
XR = prepare_matrix(X, role=RIGHT, op=N, p=p, ctx=ctx_R)
XL = prepare_matrix(X, role=LEFT, op=C, p=p, ctx=ctx_L)

# Each SCF trial
B      = gemm_prepared(H, XR)
Hraw   = gemm_prepared(XL, B)
```

skip_scalA/B を使った実装で、この動作そのものは得られた。明示的な API にする利点は、所有権・寿命・容量照会をライブラリ側で定義できることにある。一方、得られる時間は小さい。判断の目安は次のように定めていた。

- RTX 5080 で、再利用の短縮が二つの GEMM の 10 % 以上（スケーリングの割合が 20 % 以上）なら、API 提案に進む。
- それ未満なら、OpenMX 側の実装にとどめ、「fast mode では再利用が厳密に成立するが、削減は $O(n^2)/O(n^3)$ の比で小さくなる」という結果として報告する。

RTX 5080 の測定では、短縮は 4.6～6.1 %、スケーリングの割合は 10～14 % だった（第 3 節）。従って **API 提案は行わず**、OpenMX 側の実装にとどめる。準備済み表現の責任範囲（下表）は、OpenMX 側の実装が満たす条件として残す。

責任	内容	実装
準備済み表現	更新番号、形状、op、役割、moduli 数、方式が一致するときだけ使う	済
寿命と所有権	scratch とは独立に保持し、別の GEMM で上書きしない。SCF の終わりに解放	済
容量	保持量と一時領域を別々に見積もり、足りなければ同じ精度の別経路へ	済
失敗の扱い	条件不一致は再準備、容量不足は通常経路。回数を記録	済
非同期実行	準備完了・使用完了の依存関係、ストリーム間の共有	未（単一ストリーム）

8 適応精度制御と誤差監視

8.1 実装した制御

v1.2 の方針どおり、再計算と最終段階への移行を先に実装し、それを使って低精度化を評価した。制御は次の要素からなる。

- **段階列**：入力ファイルに、段階ごとの（moduli 数、スケーリング方式、次の段階へ進む残差の閾値、誤差指標の許容値）を書く。段階は上にしか進まない。

- **昇格**：SCF の残差（OpenMX が表示する NormRD）が閾値を連続 w 回下回ったら、次の段階へ進む（既定 $w = 2$ ）。停滞による昇格と、反復予算による移行も指定できる。
- **棄却と再計算**：誤差指標が許容値を超えた場合と、固有値に非数が出た場合は、その試行を捨て、同じ H_t から一つ上の段階で解き直す。密度・混合履歴に入る前なので、状態の復元は不要である。
- **最終段階**：SCF を終了できるのは最終段階だけである。最終段階は cuBLAS FP64 か、段階列の最後の GEMM8 段階かを選べる（scf.gemm8.adaptive.final）。後者ではネイティブ FP64 は棄却時の退避先としてだけ残る。
- **全ランクの同期**：全ランクが同じ大域量で同じ状態遷移を行うので、段階の配布は不要である。棄却と指標は MPI で集約する。

停止条件は対象 commit で確認した。dUele < scf.criterion かつ NormRD[0] < 10*scf.criterion で、NormRD[0] は二乗量である（表示は平方根）。

8.2 誤差監視

少数列 $b \ll n$ の試験行列 Ω について、 $Z = X\Omega$ を X が変わるまで保持し、

$$V_t = X^\dagger [H_t(X\Omega)], \quad \eta_t = \frac{\|\hat{H}_t\Omega - V_t\|_F}{\max(\|V_t\|_F, h_{\text{floor}} \|\Omega\|_F)} \quad (5)$$

を cuBLAS FP64 で計算する。v1.2 との違いは、左辺に計算されたままの $\hat{H}_t$ を使う点である（第 6.3 節）。実測で分かったことを挙げる。

- η_t は、 10^{-7} から FP64 の検出限界まで、真の相対誤差に係数 2 以内で追従する。検出限界は $3 \times 10^{-15} \sim 2 \times 10^{-14}$ ($n \approx 700 \sim 1,100$)、 $6 \times 10^{-14} \sim 7 \times 10^{-14}$ ($n = 13,000$) で、cuBLAS FP64 自身の誤差と同程度である。
- 4 方向と 16 方向で値はほとんど変わらない。既定は $b = 8$ とした。
- 費用は $n = 13,000$ 、16 方向で 3.7 ms（順変換の 1%）である。A100 では無視できる。RTX 5080 では FP64 が遅いので、測り直す。
- 段階ごとの典型値（GFRAG）は、 $L = 8$ で 2×10^{-7} 、 $L = 10$ で 9×10^{-10} 、 $L = 12$ で 5×10^{-12} 、 $L = 15$ で 2×10^{-14} である。許容値はこの 10~60 倍に置いた。 $n = 13,000$ の採取行列では指標が大きくなり（ $L = 10$ で 2×10^{-9} 、 $L = 12$ で 1×10^{-11} ）、余裕は 4~11 倍に縮む。

検査は各段階の最初と、以後 q 反復ごと（既定 $q = 5$ ）に行う。この指標は有限個の方向の検査であり、行列全体の厳密な上界ではない。

8.3 最終段階の規則の較正

較正用の三つの入力をすべて scf.criterion = 10^{-10} で実行し、 10^{-12} の FP64 計算を基準にして、最終段階の規則を比べた。fast mode、再利用ありである。段階列と昇格の閾値（NormRD）は次の四つを用意した。

名前	段階列（括弧内は次の段階へ進む閾値）
N1	$L = 10 (10^{-4}) \rightarrow 12 (10^{-7}) \rightarrow 15$ 。閾値は各段階の誤差指標の約 10^5 倍
N1a	N1 と同じ段階列で、閾値を約 10^3 倍まで詰めたもの (10^{-6} 、 10^{-9})
N2	$L = 8 (10^{-2}) \rightarrow 10 (10^{-4}) \rightarrow 12 (10^{-7}) \rightarrow 15$
N4	$L = 10 (10^{-5}) \rightarrow 14$

結果を示す。反復数は、同じ収束条件の FP64 計算（GFRAG 60 回、Mn12_148_F 61 回、Pt63 78 回）からの増減である。 $|\Delta E|$ は三つの入力での最大値で、FP64 計算自身も基準から 3.0×10^{-11} Ha 離れている。力の差はすべて 2×10^{-10} Ha/Bohr 以下であった。

名前	最終段階	履歴の 再開	連続 回数	反復数の増減			最大 $ \Delta E $ (Ha)
				GFRAG	Mn12	Pt63	
F15	固定 $L = 15$ （制御なし）	—	1	0	0	−6	3.0×10^{-11}
F12	固定 $L = 12$ （制御なし）	—	1	0	0	−5	3.2×10^{-11}
N1	最後の GEMMul8 段階	なし	1	0	0	0	3.0×10^{-11}
N1a	最後の GEMMul8 段階	なし	1	+4	0	0	2.4×10^{-11}
N2	最後の GEMMul8 段階	なし	1	0	0	+1	3.0×10^{-11}
N4	最後の GEMMul8 段階	なし	1	0	0	0	3.0×10^{-11}
N1g	最後の GEMMul8 段階	あり	2	+16	−6	+11	4.4×10^{-10}
T1	N1 のあと FP64	あり	2	+9	+7	+8	2.7×10^{-11}
T2	N1 のあと FP64	なし	2	+4	+3	+3	1.8×10^{-12}
T3	早めに FP64 (10^{-8})	なし	1	+3	0	0	2.0×10^{-11}

- **混合履歴の再開は採用しない。**反復が 7~16 回増える。さらに Mn12_148_F では、再開直後の小さな更新で停止条件が成立し、6 回早く止まって $|\Delta E| = 4.4 \times 10^{-10}$ Ha になった。目標を超える唯一の例である。v1.2 の「最終 FP64 移行時に低精度の混合履歴をクリアする」は撤回し、既定を再開なしに変えた。
- **FP64 末尾（再開なし）は 3~4 回増える。**「FP64 どうしの比較が 2 回続くまで止めない」という規則そのものの費用である。
- **早めに FP64 へ移る方式は 0~3 回増。**代わりに 4~11 回を FP64 で反復する。
- **ネイティブ FP64 を使わない方式は、閾値を指標の約 10^5 倍に置けば 0~1 回増**（N1、N2、N4）。最終段階を $L = 15$ fast にすると、全エネルギーは同じ収束条件の FP64 計算と同じ範囲に入る（GFRAG と Mn12_148_F では最後の桁まで一致）。
- $L = 8$ から始めても反復は増えない（N2）。閾値を詰めすぎると増える（N1a）。
- 誤差指標による棄却は一度も起きなかった。

- Pt63 の反復数は、設定によって 72~79 回の幅で変わる (FP64 は 78 回)。金属クラスターの収束経路は丸め程度の違いで数回変わるの、この幅の差は優劣としない。

8.4 初期ポリシー

以上から、評価に使う設定を次のように固定した。評価用入力の結果を見る前に決めたものである。

E (適応精度) 段階列 N2 ($L = 8 \rightarrow 10 \rightarrow 12 \rightarrow 15$ 、fast mode、閾値 10^{-2} 、 10^{-4} 、 10^{-7} 、連続 2 回で昇格)。最終段階は $L = 15$ で、ネイティブ FP64 は棄却時の退避先としてだけ使う。混合履歴は再開せず、通常の停止条件で終了する。誤差指標は 8 方向・5 反復ごと、許容値は 5×10^{-6} 、 10^{-8} 、 10^{-10} 、 10^{-12} 。

F (固定設定) $L = 12$ fast、再利用あり。

N2 での段階ごとの反復数は、GFRAG で $L = 8$ が 36 回・10 が 8 回・12 が 12 回・15 が 4 回、Mn12_148_F で 29・9・12・11 回、Pt63 で 35・10・18・16 回であった。反復の約半分を $L = 8$ で済ませている。

8.5 未調整入力での評価

較正に使っていない四つの入力で、E、固定設定 F ($L = 12$ fast、再利用あり)、FP64 を比べた。ポリシーは較正の結果だけで決め、評価の結果を見て変えていない。収束条件はすべて `scf.criterion` = 10^{-10} で、基準は 10^{-12} の FP64 計算である (Si1000 は計算量の都合で、 10^{-10} の FP64 を基準とした)。DIA512-1 と MCCN は、Krylov 法用の入力をクラスター解法に切り替えて使った。

入力	n	SCF 反復数			$ \Delta E $ (Ha)		
		FP64	F	E	FP64	F	E
nsV4Bz5	730	57	57	60	4.6×10^{-11}	4.5×10^{-11}	$< 10^{-12}$
DIA512-1	2,048	28	28	30	$< 10^{-12}$	$< 10^{-12}$	9×10^{-13}
MCCN	2,820	63	63	63	1.4×10^{-12}	3.2×10^{-12}	1.4×10^{-12}
Si1000	13,000	41	41	42	(基準)	1.0×10^{-11}	2.7×10^{-12}

入力	順変換 (ms、中央値)			計算全体 (秒)			E の段階別 反復数
	FP64	F	E	FP64	F	E	
nsV4Bz5	0.21	0.44	0.39	84.2	84.1	91.2	39/6/10/5
DIA512-1	1.98	2.06	1.81	39.6	41.9	41.1	15/4/9/2
MCCN	5.03	5.13	4.44	124.2	126.5	125.4	32/8/13/10
Si1000	535	309	305	1,259	1,247	1,287	14/5/16/7

段階別反復数は $L = 8/10/12/15$ の順。計算全体の時間は各 1 回の実行で、1~2 秒程度ばらつく。

- **精度**：F も E も、全入力で目標の範囲に入った。差は FP64 自身の収束の揺らぎと同程度で、力の差は 10^{-11} Ha/Bohr 以下である。誤差指標による棄却は一度も起きなかった。
- **反復数**：F はどれも FP64 と同じ反復数で止まった。E は 0~3 回増えた (nsV4Bz5 で 3 回、DIA512-1 で 2 回、Si1000 で 1 回)。較正用入力では 0~1 回だった。低精度の段階を通ると、収束の経路が少し変わる。
- **順変換**： $n \geq 2048$ では、E は F より 12~14 % 短い (中央値)。 $n = 730$ では FP64 がいちばん速い。
- **計算全体**：順変換で E が稼ぐ時間は、 $n \leq 2820$ では 1 反復あたり 1 ms 未満で、1 反復の時間 (1.4~2 秒) に比べて無視できる。反復が増えた nsV4Bz5 では、E は 8 % 遅かった。
- **Si1000**：E は 42 回で、FP64 と F より 1 回多い。段階別の順変換 (中央値) は $L = 8$ で 220 ms、10 で 284 ms、12 で 317 ms、15 で 385 ms で、42 回の半分以上を $L = 12$ と 15 で過ごしたため、1 反復あたりの順変換は F (309 ms) とほぼ同じだった。1 回多い反復の分だけ、E は計算全体で F より 40 秒 (3.3 %) 長かった。誤差指標の余裕は、 $L = 10$ で 4 倍、 $L = 12$ で 8 倍まで縮んだ。

評価の結論 (A100) 適応精度は、固定設定 F に対して順変換を 1 割強短くするが、それは計算全体の 0.1 % 程度であり、未調整入力では反復の増加 (0~3 回) のほうが大きく効いた。A100 では、E が F より速い入力はなかった。実用上の最良は、全量確保・fast mode の固定設定 ($L = 12 \sim 15$) である。RTX 5080 の第 1 回測定では、GEMM_{ul}8 の順変換が密行列解法に占める割合はさらに小さく (3 % 前後)、この結論は変わらなかった (第 3 節)。

8.6 受理・再計算の規則

事象	動作
正常な試行	密度・エネルギー・混合履歴を受理し、昇格条件を判定
η_t 超過、NaN/Inf	試行を破棄し、同じ H_t から上位段階で再計算
残差の停滞	指定した反復数だけ改善がなければ昇格 (既定では無効)
通常の停止条件を検出	最終段階でなければ終了せず、最終段階へ移行
適応段階の反復予算を超過	最終段階へ移行
最終段階でも収束しない	不収束として報告し、成功例の速度集計に入れない

棄却から再計算までの経路は、許容値を故意に厳しくした入力で確認した。棄却された試行のあと上位段階で解き直した結果は、最初からその段階で計算した結果と一致した。

9 メモリ設計と開発環境

9.1 実測した必要量

GEMMv8 v3.5.2 の作業領域の照会値を示す（実数、正方、fast mode、MiB）。「保持」は二つの役割の準備済み X の合計、「合計」は $X \cdot H \cdot B \cdot C$ の FP64 行列を含む。固有値解法の作業領域は含まない。

n	L	再利用なし		再利用あり		合計	
		作業領域		作業領域	保持	なし	あり
8,000	12	3,268		2,518	1,518	5,221	5,989
8,000	15	3,648		2,710	1,898	5,601	6,561
10,000	12	4,164		2,992	2,372	7,216	8,416
10,000	15	4,757		3,292	2,965	7,809	9,309
13,000	12	6,353		4,411	3,893	11,511	13,461
13,000	15	7,812		5,384	4,867	12,970	15,408

16 GB の RTX 5080 では、 $n = 13,000 \cdot L = 15$ の再利用は入らない見込みで、全量確保も固有値解法の作業領域と合わせると収まらず、分割実行に戻るはずである。RTX 5080 の測定 ($n \leq 7332$ 、18 ランク) では、全量確保と再利用のどちらでも容量による切替は起きなかった。ただし GPU メモリの最大使用量は 14.0 GiB (16 GB のうち) で、余裕は大きくない。 $n = 10,000$ 前後の境界は測っていない。

容量方針（実装済み） 容量不足を理由に精度を下げない。保持領域が取れなければ毎回前処理する経路へ、全量の作業領域が取れなければ分割実行へ、同じ精度のまま移る。移った回数は記録され、集計表に出る。

メモリの確保と解放そのものの費用も測った。8 GiB の確保は 1 ms 未満、解放は 4 ms で、SCF ごとに作業領域を取り直しても問題にならない。ただし、間を置かずに確保と解放を繰り返す試験では、6 GiB 以上で 1 回に数百 ms かかる場合があった（原因は未確認）。作業領域の取り直しを SCF あたり一度に抑える現在の設計は変えない。

9.2 kugui と手元環境の役割

環境	主な役割
kugui / NVIDIA A100	実装、数値回帰、SCF 制御の較正、初期プロファイリング（本版まで）
手元 / GeForce RTX 5080	主要ベンチマーク、容量制約、監視費用、収束までの総時間（第 1 回測定済み、第 3 節）

A100 で得た性能比を RTX 5080 へ移し替えない。実際に、(1) ネイティブ FP64 は RTX 5080 で A100 の 18~21 倍遅く、GEMMu8 との差が大きい。(2) 固有値解法も RTX 5080 で A100 の約 2.7 倍遅く、密行列解法の内訳が変わる（第 3 節）。RTX 5080 での測定の手順は、作業手順書 v1.1 (rtx5080_measurement_procedure_v1.1.pdf) にまとめた。

10 比較実験と対象系

10.1 比較条件

決定事項と測定結果に合わせて、比較条件を次のように改める。A 以外はすべて GEMMu8 で、対象は順変換の二つの GEMM だけである。

条件	順変換の設定	再利用	目的
A	全反復を cuBLAS FP64	対象外	基準物理量と総時間
B0	旧既定： $L = 15$ accurate、 256 MiB 分割	なし	変更前の実装（参考）
B	新既定： $L = 15$ accurate、全量 確保	なし	固定設定の出発点
B'	$L = 15$ fast、全量確保	なし	スケーリング方式の寄与
C	B' と同じ	あり	再利用単独の効果
D	適応精度（fast）	なし	適応精度単独の効果
E	適応精度（fast）	あり	提案方式の総合効果
F	許容される最速の固定設定 （fast）	あり	適応化そのものの意義

v1.2 は「B~F に同じ規則の最終 FP64 段階を設ける」とした。較正の結果（第 8.3 節）、最終 FP64 段階は必須としない。最終段階の規則は条件ごとに明記し、FP64 末尾つきの結果は別の条件として併記する。E 対 F を適応化の中心的比較、B' 対 C を再利用の比較、B0 対 B を既定変更の効果とする。F の候補は、較正用入力では $L = 12$ fast である。

10.2 FP64 基準と呼出し範囲

A は、同じ OpenMX GPU 経路・ソルバー・物理入力で、順変換の二つの GEMM だけを通常の cuBLAS FP64 にする。呼出し箇所ごとの切替は実装した (OPENMX_GEMMUL8_FORWARD=fp64)。cuBLAS 側のエミュレーションを環境変数で有効にしていないことを確認し、関連する環境変数を結果と一緒に記録する。対象外の演算（逆変換、 X の構成、固有値解法）は A~F で共通にする。

10.3 対象系と較正・評価の分離

対象	確認する性質	現状
非磁性半導体・絶縁体	基本収束、サイズ依存	測定済み (A100、RTX 5080)
金属	占有数、フェルミ準位、振動・停滞	Pt63 のみ
磁性体	磁気量、初期状態、準安定状態	Mn12_148_F のみ
非共線・SOC 入力	複素数経路、共役転置	未着手
悪条件な重なり行列	基底選別と変換誤差の感度	未着手
複数 k 点	k 点ごとの準備済み表現	未着手

較正には GFRAG、Pt63、Mn12_148_F を使った。評価用には nsV4Bz5 ($n = 730$)、DIA512-1 (2,048)、MCCN (2,820)、Si1000 (13,000) を残した (第 8.5 節)。評価結果を見て閾値を修正した場合、その入力は以後の独立評価には用いない。

10.4 三層の測定

1. **保存行列**：tests/gemmul8_reuse_probe。速度、倍々精度の参照との差、ビット比較、監視の検出能力、GEMMul8 の内部計時。合成行列でも実行できる。
2. **SCF 内部**：順変換・固有値解法・逆変換・固有ベクトル転送・誤差指標の段階別計時と、順変換の回数・前処理・再利用・容量による切替の計数。前者は環境変数 OPENMX_CLUSTER_PROFILE で、後者は OPENMX_GEMMUL8_FORWARD_TIMING で有効にする。
3. **計算全体**：tools/run_forward_variants.sh。同じ入力を設定だけ変えて実行し、反復数、全エネルギー・力・化学ポテンシャル・磁気量の差、段階ごとの反復数、棄却回数、時間をまとめる。

各実行の最初の 1 回は一度だけの費用を含むので、時間は 2 回目以降の中央値と平均で比べる。主条件は独立に 3~5 回測り、速度差がばらつき以内なら優位とは判断しない。A100 では、 $n = 13,000$ での再利用の効果 (2%) は 3 反復の実行のばらつき ($\pm 2\%$) と区別できなかった。

11 精度基準、性能モデル、再現性

11.1 最終精度の可否

目標は変えない。

$$|\Delta E| \leq 10^{-10} \text{ Ha}, \quad \max_{I,\alpha} |\Delta F_{I\alpha}| \leq 10^{-6} \text{ Ha/Bohr}. \quad (6)$$

基準は `scf.criterion` = 10^{-12} の FP64 計算とする。 10^{-10} で止めた FP64 自身が、この基準から最大 3×10^{-11} Ha 離れる (GFRAG)。全エネルギーと力に加え、化学ポテンシャル、磁気量、反復数を比較する。固有値残差と直交性の検査は未実装である。

反復数の一致は合否条件にしない。必要精度への到達時間を評価し、不収束、別の磁気状態への収束、容量超過を区別する。

11.2 再利用による削減余地

固定設定・同じ反復数 N で、対象範囲の時間に占める再利用可能な固定側 quant の割合を f 、作り直す回数を K とすると、

$$\text{speedup}_{\text{cache}} \simeq \frac{1}{1 - f + fK/N}. \quad (7)$$

実測した f (二つの GEMM に対する割合、 $L = 15$) は次のとおりである。前処理を再利用したときに消えるスケーリングの時間から求めた。

	$n = 4000$	$n = 8000$	$n = 13,000$
A100、fast mode	5.8 %	3.4 %	2.3 %
A100、accurate mode (参考)	7.3 %	4.5 %	3.0 %
RTX 5080、fast mode	6.0 %	4.6 %	—
RTX 5080、accurate mode (参考)	9.4 %	6.3 %	—

$K/N \rightarrow 0$ でも、二つの GEMM の速度向上は A100 で 1.02~1.06 倍、RTX 5080 で 1.05~1.06 倍である。密行列解法全体では、順変換の割合 (A100 で 10 % 以下、RTX 5080 で 3~4 %) を掛けて 0.5 % 以下になる。再利用は、保持に必要な容量 ($n = 13,000$ で 4.9 GiB) に見合わない。

11.3 適応精度による削減余地

二つの GEMM の時間は moduli 数にほぼ比例する ($n = 13,000$ で約 23 ms/個)。反復の内訳が分かれば、順変換の時間は段階ごとの反復数から見積もれる。較正で得た段階ごとの反復数 (第 8.3 節) に $n = 13,000$ の時間を当てはめると、1 反復あたりの順変換は、E (N2) で 222~246 ms、固定 $L = 12$ で 276 ms、固定 $L = 15$ で 346 ms になる。E は F より 11~20 %、固定 $L = 15$ より 29~36 % 短い。密行列解法 (約 4.2 秒) に対しては、それぞれ 0.7~1.3 %、2.4~3.0 % である。ただし Si1000 での実測では、E は反復の半分以上を $L = 12$ と 15 で過ごし、1 反復あたりの順変換 (305 ms) は F とほぼ同じだった (第 8.5 節)。

全体に対する比率をまとめる。 $n = 13,000$ 、A100 の値である。

変更	順変換の時間	密行列解法に対する差
旧既定 ($L = 15$ accurate、分割) → 新既定 (全量確保)	810 → 421 ms	−8.3 %
新既定 → $L = 15$ fast	421 → 391 ms	−0.7 %
$L = 15$ fast → 再利用	391 → 370 ms	−0.5 %
$L = 15$ fast → $L = 12$ fast (固定、再利用)	−20 % (行列単体)	−1.7 %
$L = 12$ fast → E (見積り)	−11~−20 %	−0.7~−1.3 %

既定の変更が支配的で、残りは合わせても密行列解法の数 % である。固有値解法 (3.6 秒) が動かない限り、順変換だけを対象にした改良の上限はここにある。さらに、最後の行の利点は反復 1 回分 ($n = 13,000$ で約 31 秒) より小さい。評価用入力では E が反復を 0~3 回増やしたので (第 8.5 節)、この利点は消える。RTX 5080 では固有値解法がさらに大きな割合を占めるので、順変換の改良の上限は A100 より小さい (第 3 節)。

11.4 保存する情報

ソース commit、依存ライブラリ、コンパイラ、GPU、MPS の状態、関連する環境変数、入力、制御設定、各反復の段階・切替理由・受理状態、準備回数、再利用回数、検査値、容量による切替、時間内訳、最終物理量を、実行ごとの `environment.txt`・標準出力・集計表に残す。測定用のスクリプトがこれを自動で行う。

12 工程と判断点

12.1 進み具合

期間	作業	状況
1-2 週	版・形状・スケール依存性の監査、行列採取、FP64 基準	完了 (A100)
3-4 週	固定精度の再利用、状態保存・FP64 復帰	共線クラスター経路で完了。明示的 API は提案しない
5-6 週	moduli 制御、誤差監視、再試行、較正	実装完了。較正 3 入力、評価 4 入力 (第 8.5 節)
7-8 週	fast mode、複素数・磁性、容量、RTX で予備測定	fast mode は主設定に決定。複素数は未着手。RTX で第 1 回測定 (第 3 節)
9-10 週	RTX 5080 で A-F を反復測定	固定設定のみ測定済み。続けるかは第 12.2 節の判断による
11-12 週	解析、論文図表、API 提案準備	API 提案は行わない (第 7.3 節)。まとめは未着手
10 月 6～7 日	固有値解法の FP32 化と補正 (第 4 節)	非共線クラスターで実装・検証 (A100、RTX 5080)。共線クラスター、共線バンド、非共線バンド (1 ランク 1k 点と複数 k 点の両経路) へ拡張し A100 で検証。RTX 5080 の共線クラスターは測定済み、バンドは依頼中

12.2 今後の方針 (提案)

v1.3 の方針 1～3 (全量確保を成果として確定、再利用 API は提案しない、適応精度は打ち切る) はそのままとし、方針 4 の対象を次のように具体化する。

- 固有値解法**：補正つき FP32 固有値解法 (第 4 節) を成果として確定する。RTX 5080 での共線クラスター・バンドの測定を受け取り、A100 の精度検証と合わせて反復数・全エネルギー・力の差を表にする。
- バンドの複数 k 点経路**：実装と検証を終えた (第 4 節)。ウォームスタートの偏りと SCF 反復の増加の原因 (補正しない列の古い成分) は特定して修正した (第 4 節)。
- 格子上の処理**：第 5 節のとおり、軌道値のその場計算と double-float カーネルを実装し、RTX 5080 での測定を残すのみである。以下は着手前の検討の記録である。RTX 5080 で全体の 6 割を占める

Set_Hamiltonian (1 ステップ 3.8 秒) と Set_Density_Grid を対象にする。原因は GPU メモリ不足による CPU への退避 (第 4 節の制限の項) なので、精度ではなくメモリ設計の問題である。候補は、(a) 既存の直列ウェーブ方式 (OPENMX_SETHAM_GPU_SERIAL_WAVES=1: 全ランクが順番に GPU を使う)、(b) ストリーミング方式 (表を小さな束で送りながら計算する。現状は 1 ランクしか入らないときだけ選ばれるので、入りきらないとき全般に広げる)、(c) 軌道の格子値を GPU 上でその場で再計算する方式 (表を持たない。FP32 の軌道値は現状の Type_Orbs_Grid と同じ精度)。A100 で GPU メモリの上限を模擬して比べた結果 (sidia333 非共線、8 ランク、12 反復の Set_Hamiltonian の時間) は、上限なし 17.4 秒、上限ありのハイブリッド (既定。2~3 ランクが GPU、残りが CPU) 165 秒、直列ウェーブ 37.6 秒、ストリーミング 52.3 秒で、直列ウェーブが既定の 4.4 倍速かった。RTX 5080 では OPENMX_SETHAM_GPU_SERIAL_WAVES=1 の測定を手順書に加えた (変種 DW)。

4. **成果のまとめ**: GEMM の再利用・適応精度の結果 (v1.3) と固有値解法の補正 (v1.4) を一つの技術報告にまとめる。主張は「GeForce 級の GPU で、FP64 の固有値解法を FP32 解法 + GEMMul8 による反復改良で置き換え、SCF の精度を保ったまま全計算を 25% 短縮した」とし、適用条件 (FP32:FP64 比、GEMMul8 の精度、縮退クラスターの扱い) を明記する。

12.3 段階ごとの判断

観測結果	対応
固定側 quant が小さい	両 GPU で観測した (A100 で 2~6 %, RTX 5080 で 10~14 %)。API 提案は行わない
特定 mode で独立準備が成立しない	accurate mode で観測した。fast mode に対象を限定した
キャッシュで 16 GB を超える	$n = 13,000 \cdot L = 15$ で超える見込み (RTX 5080 では $n \leq 7332$ だけ測定)。同じ精度の別経路へ戻る
監視・最終 FP64 の費用が大きい	監視は小さい。最終 FP64 は反復を 3~4 回増やすので、既定では使わない (最後の GEMMul8 段階を最終とする)
E が F より速くない	A100 の評価用入力で観測し、RTX 5080 でも余地がないことを確認した。固定設定を結論とする
磁気状態・収束先が変わる	これまで観測していない。失敗を分類し、条件を満たす例だけで速度を比較
補正後も全エネルギーが FP64 と 10^{-10} Ha 以上違う	観測していない (第 4 節)。起きた場合は改良回数を 3 にし、クラスタの占有数判定を緩める
縮退クラスタが大きすぎてレイリー・リッツが遅い	δ の固定で解消。上限 ($\min(2048, \sqrt{n \text{ MaxN}/2})$) を超えたステップは FP64 に戻る
ウォームスタートの拒否が続く	残差 2×10^{-5} 超のステップは FP32 で解き直す (A100 の sidia333 で 25 回中 5 回、いずれも初期のステップ)
GPU メモリ不足でターンが直列化される (バンド)	補正は使わず FP64。k 点数を減らすか、複数 k 点経路の退避方式 (方針 2) を待つ

12.4 論文に必要な図表

1. moduli 数と達成精度・時間の関係 (実行列、FP64 との比較)。
2. 再利用の成立条件 (スケーリング方式) と、行列サイズに対する削減率・保持容量。
3. 作業領域の確保方針 (分割・全量) による時間の違いと、その内訳。
4. SCF 残差・監視値・精度段階の履歴と、最終段階の規則ごとの反復数。
5. A-F の総時間と内訳、最終物理量の差、FP64 基準の変動。
6. 補正つき FP32 固有値解法：改良回数と固有値・密度行列の誤差 (探針)、SCF の反復数と全エネルギー・力の差 (A100、RTX 5080)、密行列解法 1 回あたりの内訳 (FP64、FP32 解法、改良、レイリー・リッツ、密度行列)、ウォームスタートの採用率。

7. RTX 5080 の全計算の内訳 (Set_Hamiltonian、Set_Density_Grid、対角化、力) と、固有値解法の置き換え前後の比較。

「常に高速」「有効ビット数だけで精度を保証」「再利用や SCF 適応精度の初出」は主張しない。

13 v1.3 からの主な改訂点

項目	v1.4 での加筆・修正
中心仮説の扱い	v1.2 の仮説と v1.3 の結論を冒頭に要約し、第 2 段階 (固有値解法) の結果を加えた
研究質問	RQ4 (固有値解法の精度を下げて SCF の精度を保てるか) を追加
固有値解法	第 4 節を新設。案 A・B の比較、荻田・相島の反復改良と占有数によるクラスタの扱い、実装 (共有モジュール、二段階 FP32 解法、ウォームスタート、段階制御、既定値、密度行列の積化)、A100 の検証、RTX 5080 の測定、制限
他の経路	共線クラスター、共線バンド、非共線バンドへの拡張とその検証結果を記載
今後の方針	対象を固有値解法から、バンドの複数 k 点経路と格子上の処理 (Set_Hamiltonian、Set_Density_Grid) へ具体化
段階ごとの判断・図表	補正つき固有値解法に関する行と図表を追加
参考文献	荻田・相島の反復改良 (2018、2019) を追加

v1.2 から v1.3 への改訂点は v1.3 の第 9 節にある。本版の測定値は、kugui (A100、CUDA 12.5、cuSOLVER 11.6、GEMMv8 v3.5.2) と手元の PC (RTX 5080、CUDA 13.4) でブランチ adaptive-precision を用いて得た。生データと集計は work/bench_adaptive の記録 (eigcol1_20261007_results.md、rtx5080_nc_20261007/, eigoa_20261006/ など) にある。

参考文献

- [1] RIKEN R-CCS, *GEMM8: GEMM emulation and its extension to BLAS-like matrix operations using INT8/FP8 matrix engines*, public README.
<https://github.com/RIKEN-RCCS/GEMM8>
参照箇所: Scaling Modes, Accuracy Guidelines, Complex-valued operations, Return value, Workspace query, Memory-saving mode, Behavior of skip_scalA / skip_scalB. 参照日: 2026 年 10 月 5 日。本版の測定はタグ v3.5.2 (commit 603b523) による。
- [2] K. Ozaki, Y. Uchino, and T. Imamura, *Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique*, The International Journal of High Performance Computing Applications (2026), OnlineFirst (2026 年 7 月 28 日公開) .
<https://doi.org/10.1177/10943420261467787>
本文は arXiv:2504.08009v4 (2026 年 9 月 16 日改訂) を参照。特にスケーリング、CRT 復元、入力分布と達成精度の関係。
- [3] H. Huang, W. Shao, and J. Hammond, *Accelerating Density Fitting with Adaptive Precision and 8-Bit Integer on AI Accelerators*, The Journal of Physical Chemistry A **130**(17), 3483–3493 (2026).
<https://doi.org/10.1021/acs.jpca.6c00225>
本文は arXiv:2601.08077v3 (2026 年 4 月 17 日改訂) を参照。適応精度、単調昇格、FP64 への復帰、他ライブラリでの実装に関する節。
- [4] T. Ogita and K. Aishima, *Iterative refinement for symmetric eigenvalue decomposition*, Japan Journal of Industrial and Applied Mathematics **35**, 1007–1035 (2018).
<https://doi.org/10.1007/s13160-018-0310-3>
対称固有値分解の反復改良。残差 $R = I - X^\dagger X$, $S = X^\dagger A X$ から補正 E を作り、1 回の改良で誤差が 2 乗のオーダーになること。
- [5] T. Ogita and K. Aishima, *Iterative refinement for symmetric eigenvalue decomposition II: clustered eigenvalues*, Japan Journal of Industrial and Applied Mathematics **36**, 435–459 (2019).
<https://doi.org/10.1007/s13160-019-00348-4>
縮退・近接固有値のクラスタに対する扱い。本研究のクラスタの判定 (δ 、連なりの幅) とレイリー・リッツはこれに基づく。
- [6] OpenMX, *User's manual of OpenMX Ver. 4.0*, Section 16.1, SCF convergence basics.
https://openmx-square.org/openmx_man4.0/s16_1_scf-convergence-overview.html
参照日: 2026 年 10 月 5 日。NormRD と scf.criterion の説明を参照。実際の停止条件は対象 commit で確認した (第 8.1 節)。
- [7] H. Kawai (dc1394), *openmx4.0_gpu*, public README.
https://github.com/dc1394/openmx4.0_gpu/blob/main/README.md
参照日: 2026 年 10 月 5 日。gpusolver 経路と GEMM8 の利用・切替を参照。本研究の実装はブランチ adaptive-precision にある。
- [8] NVIDIA, *GeForce RTX 5080 Graphics Cards*, official specifications.
<https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5080/>
参照日: 2026 年 10 月 5 日。標準メモリ構成 16GB GDDR7。

TeX ソースの再コンパイル

UTF-8 の本ソースを、日本語対応の TeX Live (LuaTeX-japan を含む) でコンパイルする。原ノ味フォントがなければ IPAex 明朝・ゴシックを使う。参考文献はソース内に含み、外部画像や BibTeX ファイルは不要。

```
lualatex adaptive_precision_research_plan_v1.4.tex
```

相互参照と総ページ数を確定するため、同じコマンドを 2 回実行する。